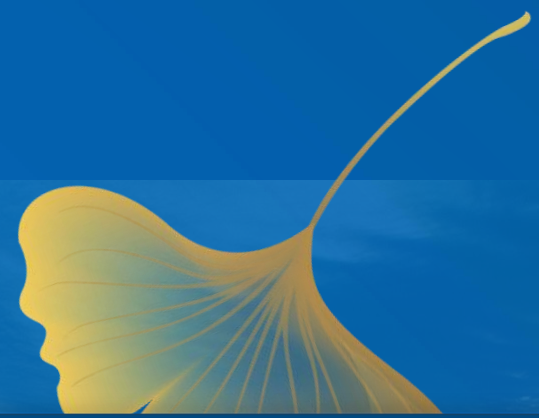




南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

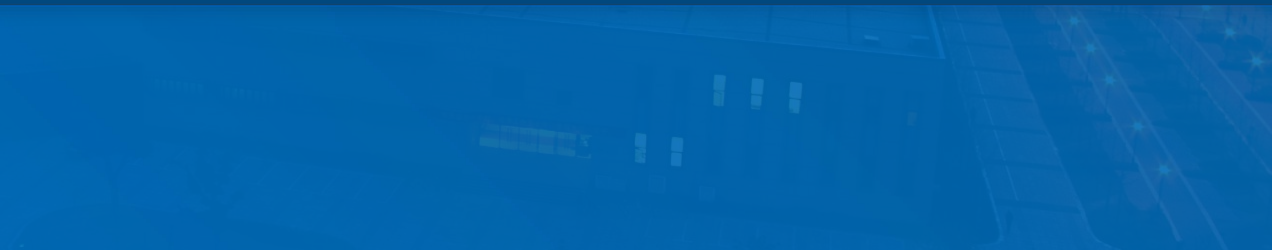
1902 2022



Designing Software Architectures

A Practical Approach

李杉杉





南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

About Me



李杉杉
特聘研究员

李杉杉，南京大学软件学院准聘助理教授 (特聘研究员)， 软件开发效能实验室成员。博士毕业于南京大学软件学院，曾在澳大利亚阿德莱德大学接受联合培养一年，荣获2022年江苏省双创博士。主要研究方向为微服务架构、软件架构智能化、区块链与智能合约等，具体研究内容包括智能化微服务架构设计与演进、面向区块链的软件架构设计、区块链性能仿真与评估、区块链和微服务软件质量保证、智能合约可定制化设计及DevOps支持技术等。在本领域TSE、SPE、软件学报等一流期刊和ISSTA、FSE等权威国际会议上发表论文20余篇，其中参与发表的2篇论文分别获得APSEC 2016最佳论文奖、APSEC 2023最具影响力论文奖，申请发明专利15项 (获得授权4项)，作为主要起草人参与制定国家标准一项 (实施中)。主持一项国家自然科学基金青年基金项目，参与多项国家级、省部级课题项目。



个人主页: software.nju.edu.cn/lss



联系邮箱: lss@nju.edu.cn



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

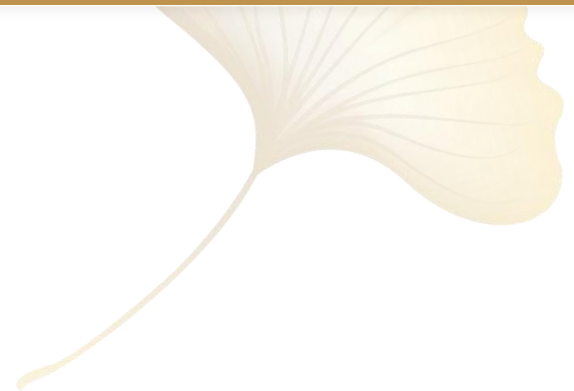
CONTENT

- ◎ **Architectural Drivers and Design Strategies**
- ◎ **Attribute–Driven Design Method (ADD 3.0)**
- ◎ **Documenting Software Architectures**
- ◎ **Case Study: Hotel Pricing System**
- ◎ **Summary**





Architectural Drivers and Design Strategies

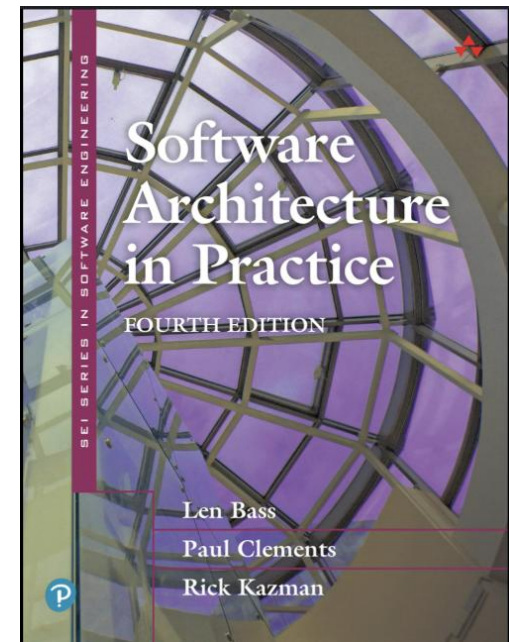


Software Architecture

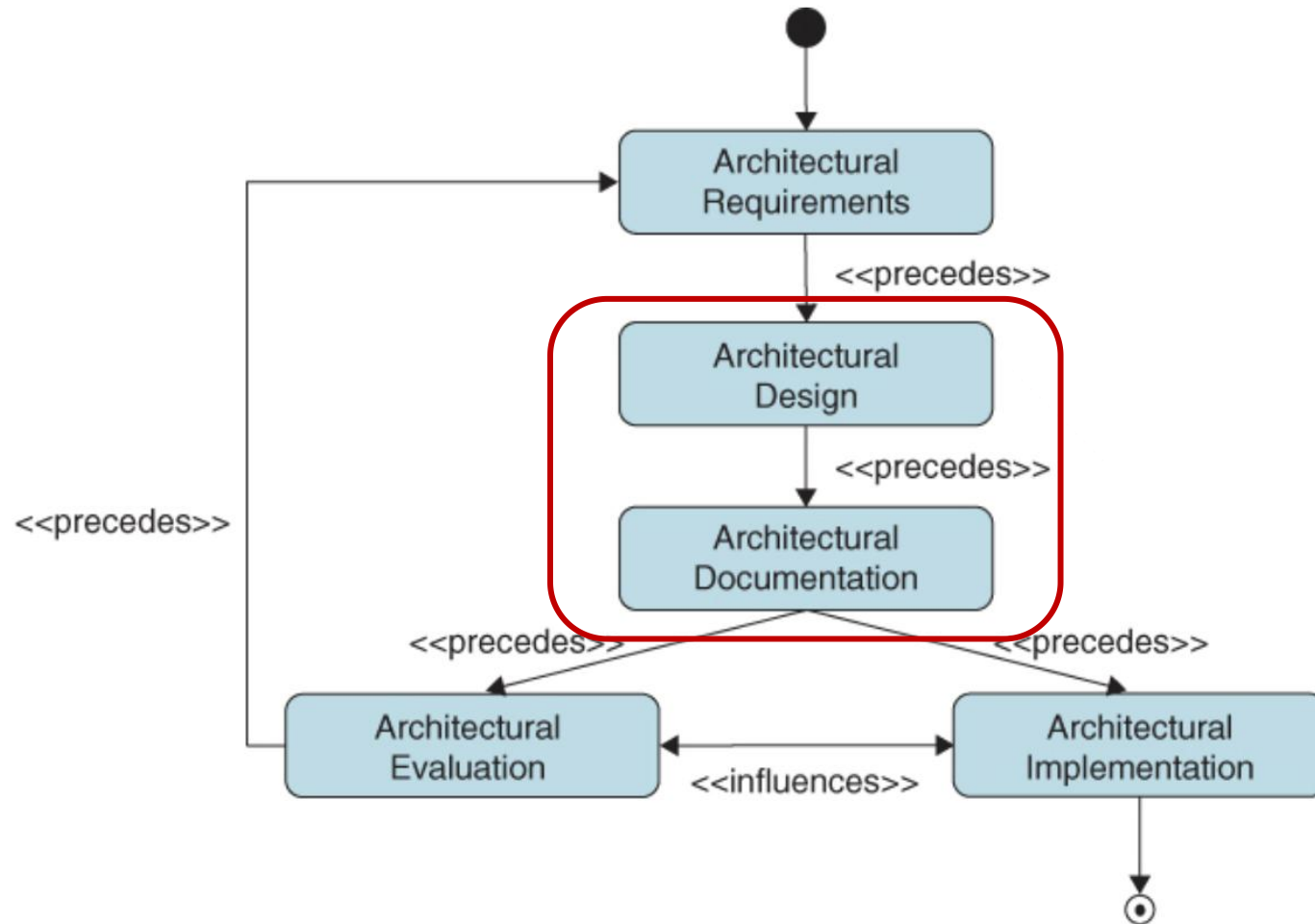
What is it?

- The software architecture of a system is the set of **structures** needed to reason about the system.
- These structures comprise **software elements, relations** among them, and **properties** of both.

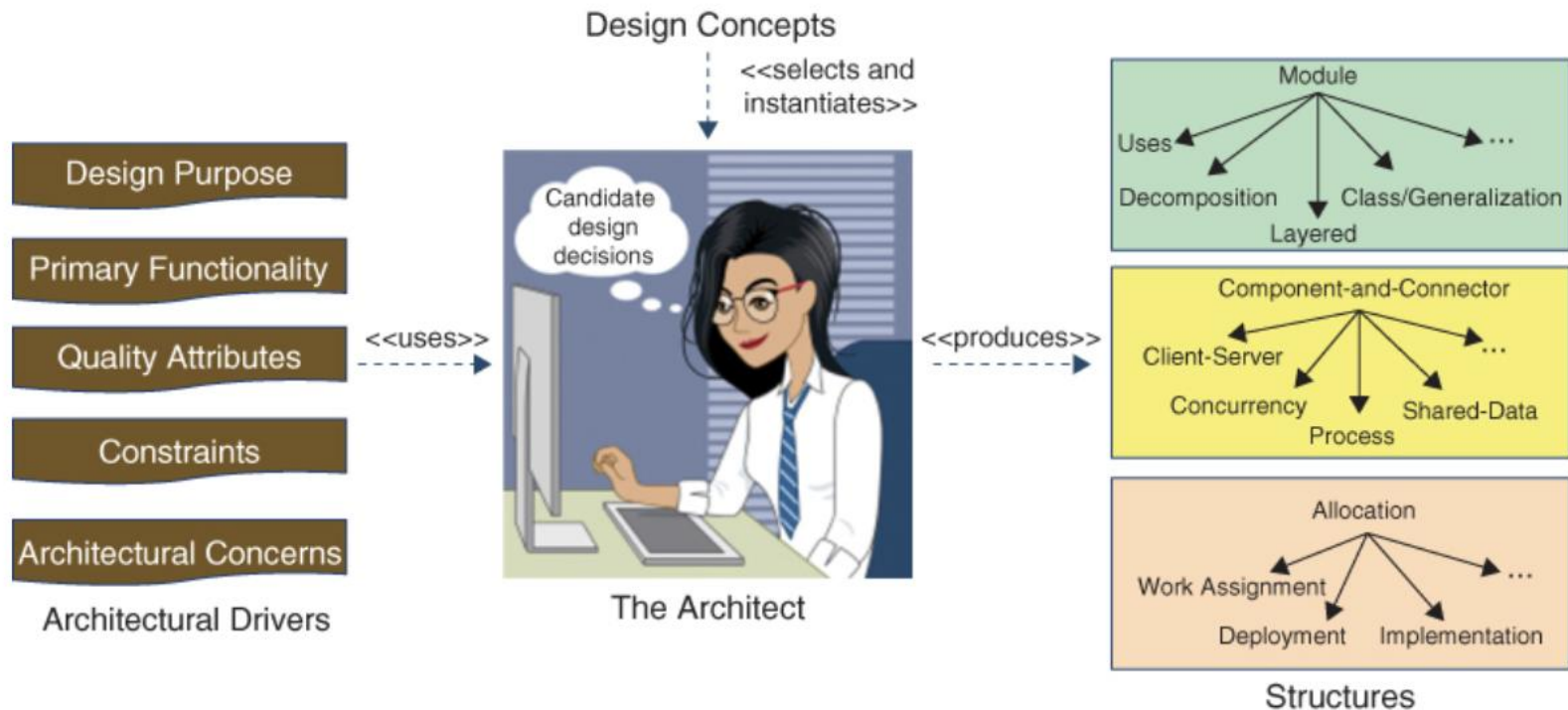
- «*Software Architecture in Practice*»
(fourth edition)



Software Architecture Life-cycle Activities



Architectural Design Activity



Architectural drivers

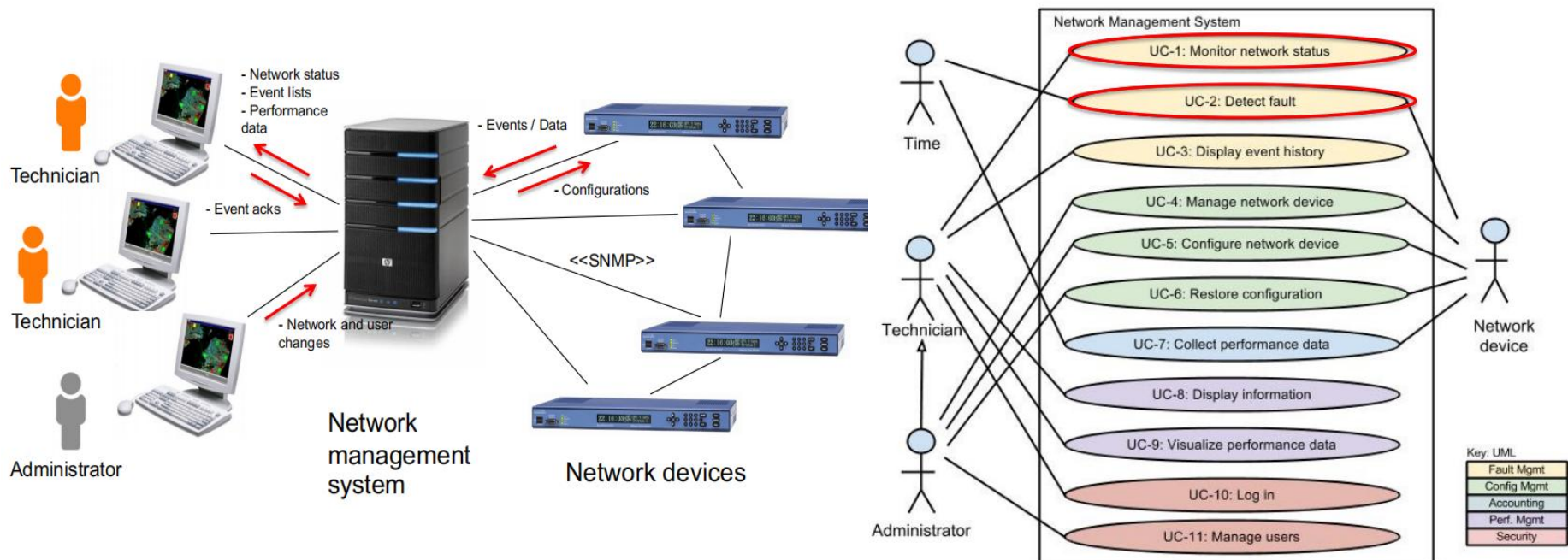
Categorize those “what” and “why” questions

- They are a **subset of the requirements** that shape the architecture
 - Functional requirements
 - Quality attribute requirements
 - Constraints
- But **other drivers** include:
 - The type of system that is being designed
 - Design objectives
 - Concerns
- These are the **inputs** to the design process

Architectural drivers

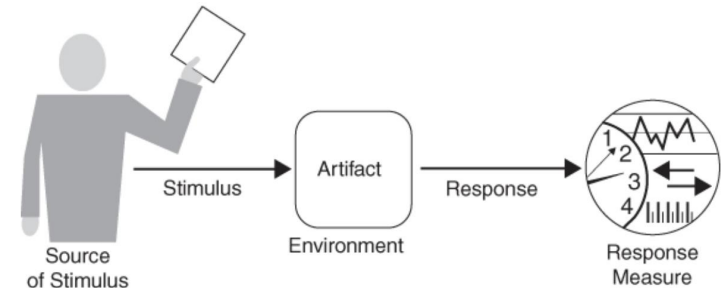
Functional drivers

- Typically involve **primary functionality**, i.e. functionality that directly supports the **business goals**
 - Example: Network Management System “Marketecture”

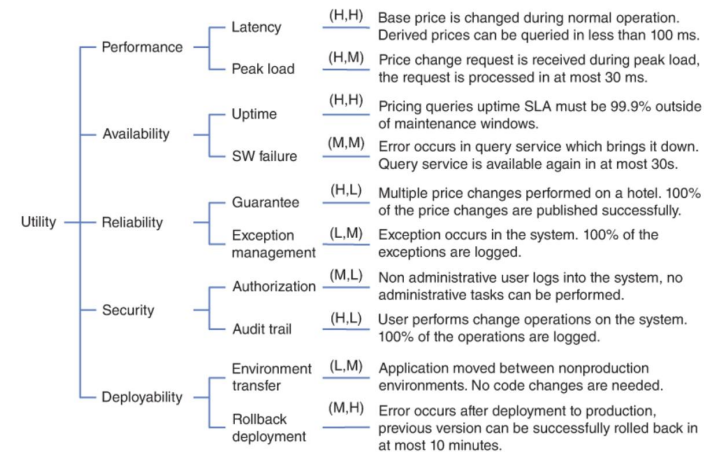


Architectural drivers

Quality attribute drivers



- Measurable characteristics of interest to users and developers
 - Performance, Availability, Modifiability, Testability, etc...
 - Can be specified using the scenario technique
 - Prioritized by the customer according to importance to the success of the system (H, M, L) and by the architect according to technical risk (H, M, L)



BUSINESS IMPORTANCE/TECHNICAL RISK	L	M	H
L	5, 6, 17, 20, 22	1, 14	12, 19
M	9, 12, 16	8, 20	3, 13, 15
H	10, 18, 21	4, 7	2, 11

Architectural drivers

Quality Attribute Drivers

ID	Quality Attribute	Scenario	Associated use case	Priority
QA-1	Performance	Several network devices send traps to the management system at peak load. 100% of the traps are successfully processed and stored.	Detect network device fault (UC-2)	H, H
QA-2	Modifiability	A new network device management protocol is introduced to the system as part of an update. The protocol is added successfully without any changes to the core components of the system.	Configure network device (UC-5)	M, M
QA-3	Availability	A failure occurs in the management system during operation. The management system resumes operation in less than 30 seconds.	All	H, H
QA-4	Performance	The management system collects performance data from a network device during peak load. The management system collects all performance data within 5 minutes to ensure no loss of data.	Collect performance data (UC-7)	H, H
QA-5	Performance, Usability	A user displays the event history of a particular network device during normal operation. The list of events from the last 24 hours is displayed within 1 second.	Display Event history (UC-3)	H, M
QA-6	Security	A user performs a change in the system during normal operation. It is possible to know who performed the operation and when it was performed 100% of the time.	All	H, M

Quality Attributes Scenarios of the Network Management System “Markecture”

Architectural drivers

Constraints



- System design also has "constraints" that shape the architecture, just like building a house, where budget and land area limit your design (e.g., no villa with a tight budget).
 - **Time & Budget**
 - Impact: A 3-month project deadline rules out complex modules requiring 6 months of development.
 - Example: Prioritize minimal viable product (MVP) features over elaborate functionalities.
 - **Technical Limitations**
 - Impact: Lack of blockchain expertise in the team? Avoid adding "blockchain-based proof" sub-requirements.
 - Risk: Skill gaps can lead to implementation delays or failures.
 - **Business Rules & Regulations**
 - Mandate: E-commerce systems must comply with GDPR for data privacy.
 - Action: Embed "compliance checks" in every user data-related module during architectural design.

Architectural drivers

Constraints



constraints

ID	Constraint
CON-1	A minimum of 50 simultaneous users must be supported.
CON-2	User workstations use the following operating systems: Windows, OSX, and Linux.
CON-3	An existing relational database server must be used.
CON-4	Network connection between users workstations and the server is unreliable
CON-5	Future support for mobile clients
CON-6	A minimum of 600 time servers must be supported (Initial deployment 100 time servers, 100 more / year during 5 years)
CON-7	Each time server sends, on average, 10 traps/hour.
CON-8	Performance data needs to be collected in intervals of no more than 5 minutes as higher intervals result in time servers discarding data.
CON-9	Events from the last 30 days must be stored
CON-10	The development team is familiar with Java technologies

Constraints of the of the Network Management System “Marketecture”

Architectural drivers

Types of systems

- Greenfield systems in novel domains
 - E.g. Google, Amazon, WhatsApp
 - Less well known domains, more innovative
- Greenfield systems in mature domains
 - E.g. “Traditional” enterprise applications,
 - standard mobile applications
 - Well known domain, less innovative
- Brownfield systems
 - Changes to existing system



Architectural drivers

Design Purpose/Objectives

- Before you can begin you need to be clear about **why you are designing**. Your objectives will change what and how you design.
 - For a **pre-sales proposal**, which usually involves the rapid design of an initial solution in order to produce an estimate
 - For a **custom system** with established time and costs and which may not evolve much once released
 - For a **new increment or release** of a continuously evolving system



Architectural drivers

Architectural Concerns

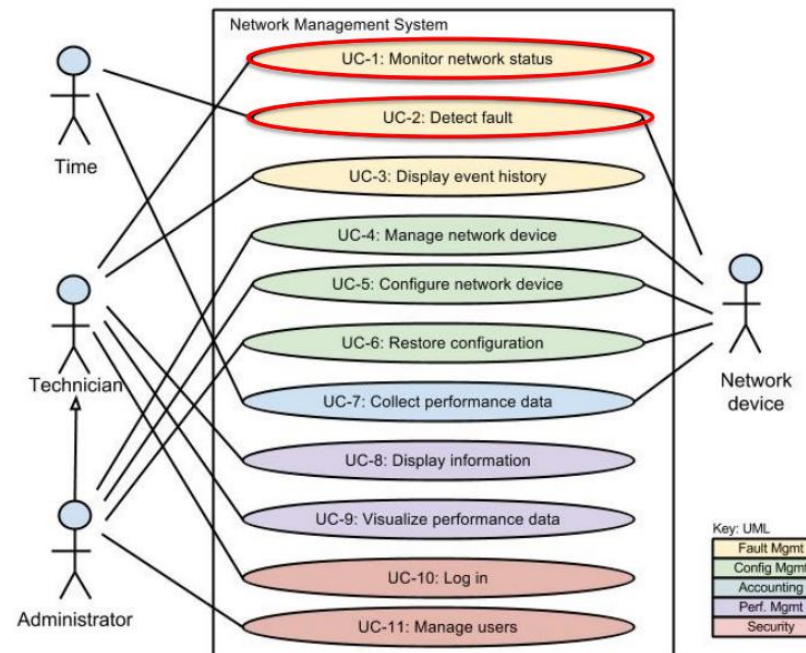
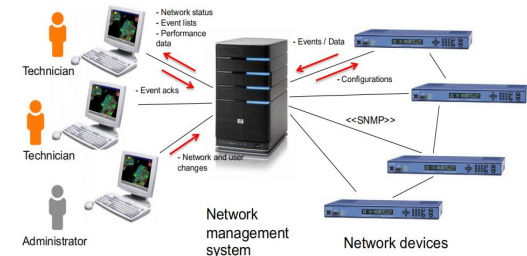
- Additional aspects that need to be considered as part of architectural design but are **not expressed as traditional requirements**.
 - **General Concerns**: “**broad**” issues, the overall system structure, functionality->modules, modules->teams, & code base organization.
 - **Specific Concerns**: more **detailed, system-internal** issues (result from previous design decisions), i.e. logging, API versioning .
 - **Internal Requirements**: though not explicitly from customers, are vital. They can **impact component design and interaction**, e.g., specific encryption for data security influencing tech choices.
 - **Issues**: like security **risks** & performance **bottlenecks** require architectural changes.



Architectural drivers

Types, Design Objectives, and Concerns of the Example

- Type Of System:
 - Greenfield in mature domain
- Objective:
 - Design in preparation for construction of an increment of the system
- Concerns
 - Structure the system
 - Organization of the codebase
 - Input validation
 - Exception management
 - ...



Design Strategies

Decomposition

- Quality attribute drivers can be decomposed and assigned to the elements of the decomposition.
 - Split large system goals into manageable sub-tasks, like solving a 1,000-piece puzzle by dividing it into "sky," "mountain," and "figure" sections.
 - Purpose: Make abstract requirements actionable by assigning clear responsibilities to modules/components.
- Keep in mind the constraints given and arrange the decomposition so that it will accommodate those constraints .
- The goal of the design activity is to generate a design that accommodates the constraints and achieves the quality and business goals for the system.

Design Strategies

Designing to ASRs

- What about the non-ASR requirements?
 - The choice of ASRs implies a **prioritization** of the requirements.
 - A) You **can still meet** the other requirements.
 - B) You can meet the others with a **slight adjustment** of the existing design.
 - C) You **cannot meet** the others under the current design.
 - a) you are **close** to meeting the requirements.
 - b) **reprioritize** the requirements and **revisit** the design.
 - c) you **cannot meet** requirements.
- Design for all of ASRs or one at a time?
 - The answer is a matter of **experience**.
 - Through experience and education, you will develop an intuition for designing, and employ **patterns/tactics** to aid you in designing for multiple ASRs.

Design Strategies

Generate and Test

- View a particular design as a hypothesis: the things wrong with the current design hypothesis are fixed in the next design hypothesis, and the things right are kept.
 - Where does the initial hypothesis come from?
 - Existing systems
 - Frameworks (partial designs)
 - Patterns and tactics
 - Design checklists (providing guidance and confidence)
 - What are the tests that are applied?
 - Analysis techniques
 - Design checklists
 - How is the next hypothesis generated?
 - When are you done?
 - Either have a design that satisfies the ASRs or when you exhaust your budget for design.
 - Implement the best hypothesis you made

Design Decisions

The core of the design process

- The design process involves different making design decisions
 - Selecting elements to decompose
 - Choosing one or more design concepts that satisfy the inputs considered in the iteration
 - Instantiating architectural elements, allocating responsibilities and defining interfaces
- Selecting design decisions can be particularly challenging...



Design Concepts

Avoid reinventing the wheel

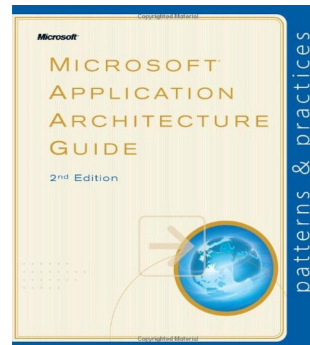
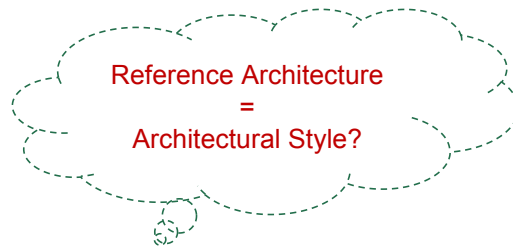
- Most sub-problems that are addressed during an iteration can be solved using **existing solutions**, i.e. design concepts
 - Architects are problem solvers, not inventors.
 - Avoid "reinventing the wheel": **No need to redesign proven solutions** (e.g., user permissions, distributed caching).
 - Data shows: Using validated approaches typically leads to **Better, Faster, Lower Risk** designs.
 - **Creativity in design involves Choosing wisely, Combining smartly and Evolving flexibly**
- There are several categories of design concepts, some are abstract and some more concrete. Here we consider:
 - **Reference Architectures**
 - **Deployment Patterns**
 - **Architectural / Design Patterns**
 - **Tactics**
 - **Externally developed components** (e.g. Frameworks)



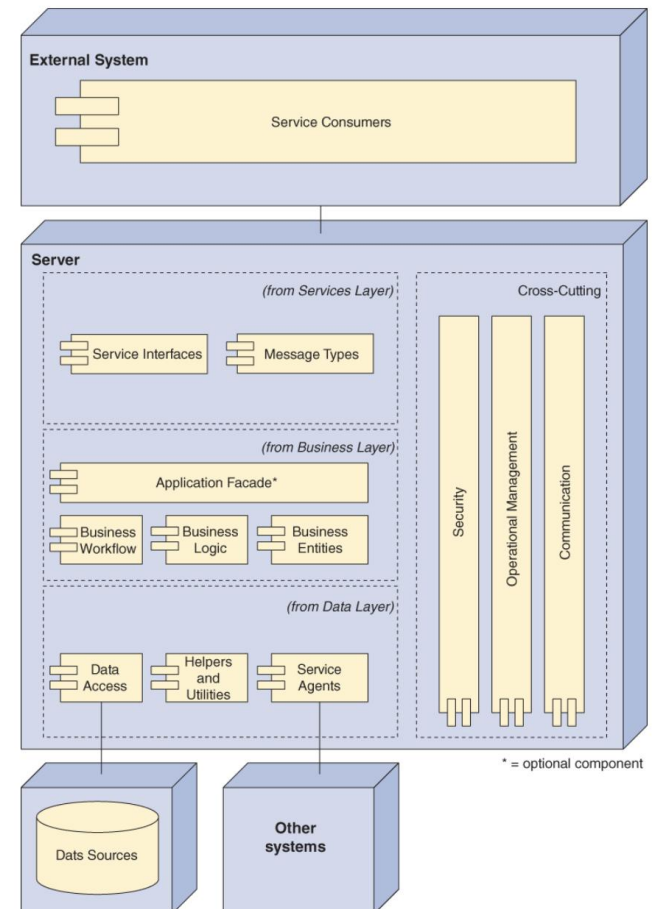
Design Concepts

Reference Architectures

- They provide a **blueprint for structuring** an application. Examples for the enterprise application domain include
 - Mobile applications
 - Rich client applications
 - Rich internet applications
 - Service Applications
 - Web applications



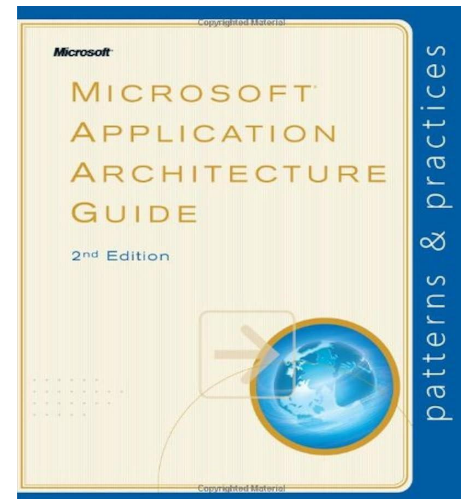
拓展阅读：《Application Architecture Guide》



Design Concepts

Deployment Patterns

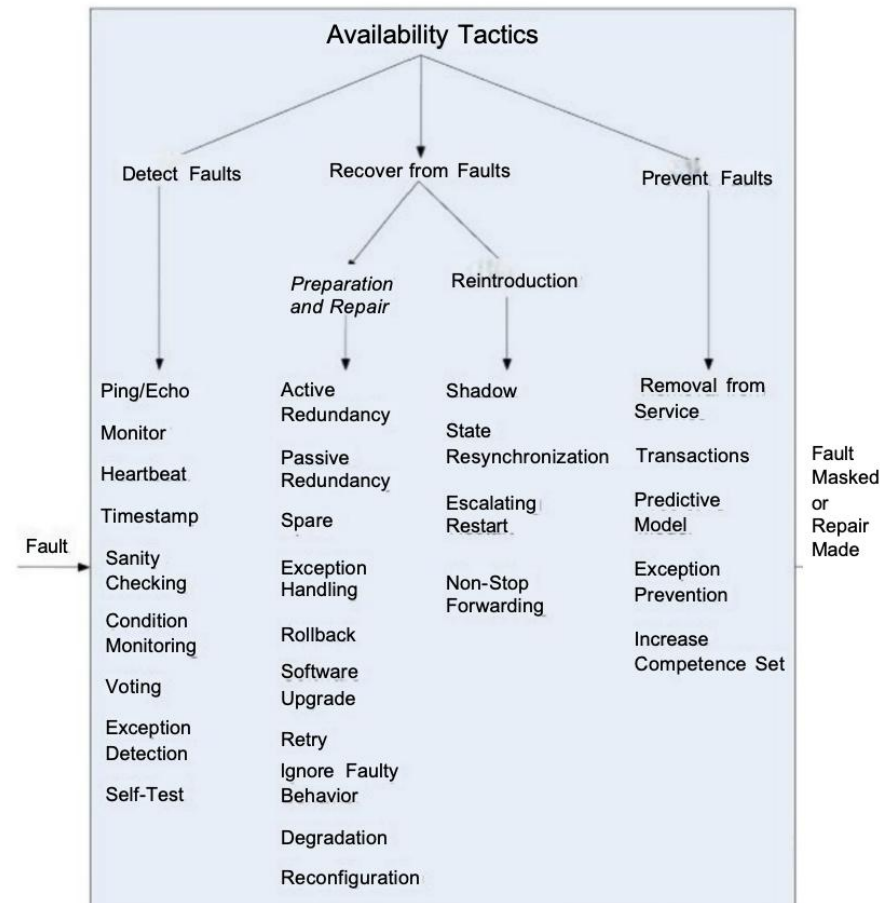
- Deployment patterns provide guidance on how to **structure the system from a physical standpoint**.
- Good decisions with respect to the deployment of the software system are essential to achieve quality attributes such as availability.
- Examples
 - 2, 3, 4 and n-tier deployment
 - Load balanced cluster
 - Failover cluster
 - Private/public cloud
 - Etc...



Design Concepts

Tactics

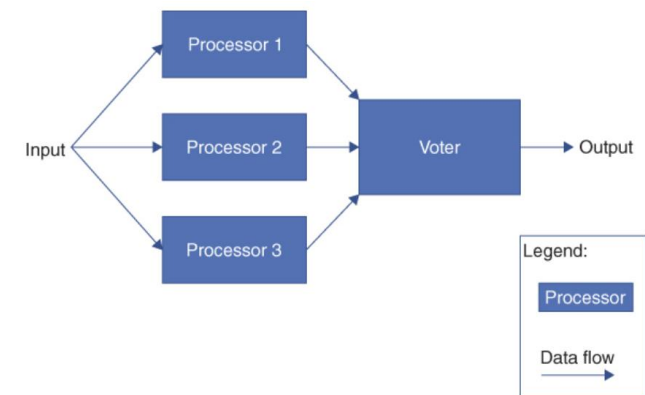
- What are tactics?
 - Design decisions that influence the control of a quality attribute response.
- There are tactics categorizations for the quality attributes of:
 - Availability
 - Interoperability
 - Modifiability
 - Performance
 - Security
 - Testability
 - Usability



Design Concepts

Patterns

- Patterns are proven (conceptual) solutions to recurring design problems. Originated in building architecture.
- Many patterns exist (thousands), and they are documented across several pattern catalogs.
- It is difficult to draw a clear boundary between “design” and “architectural” patterns.



Tri-modular redundancy patter

Design Concepts

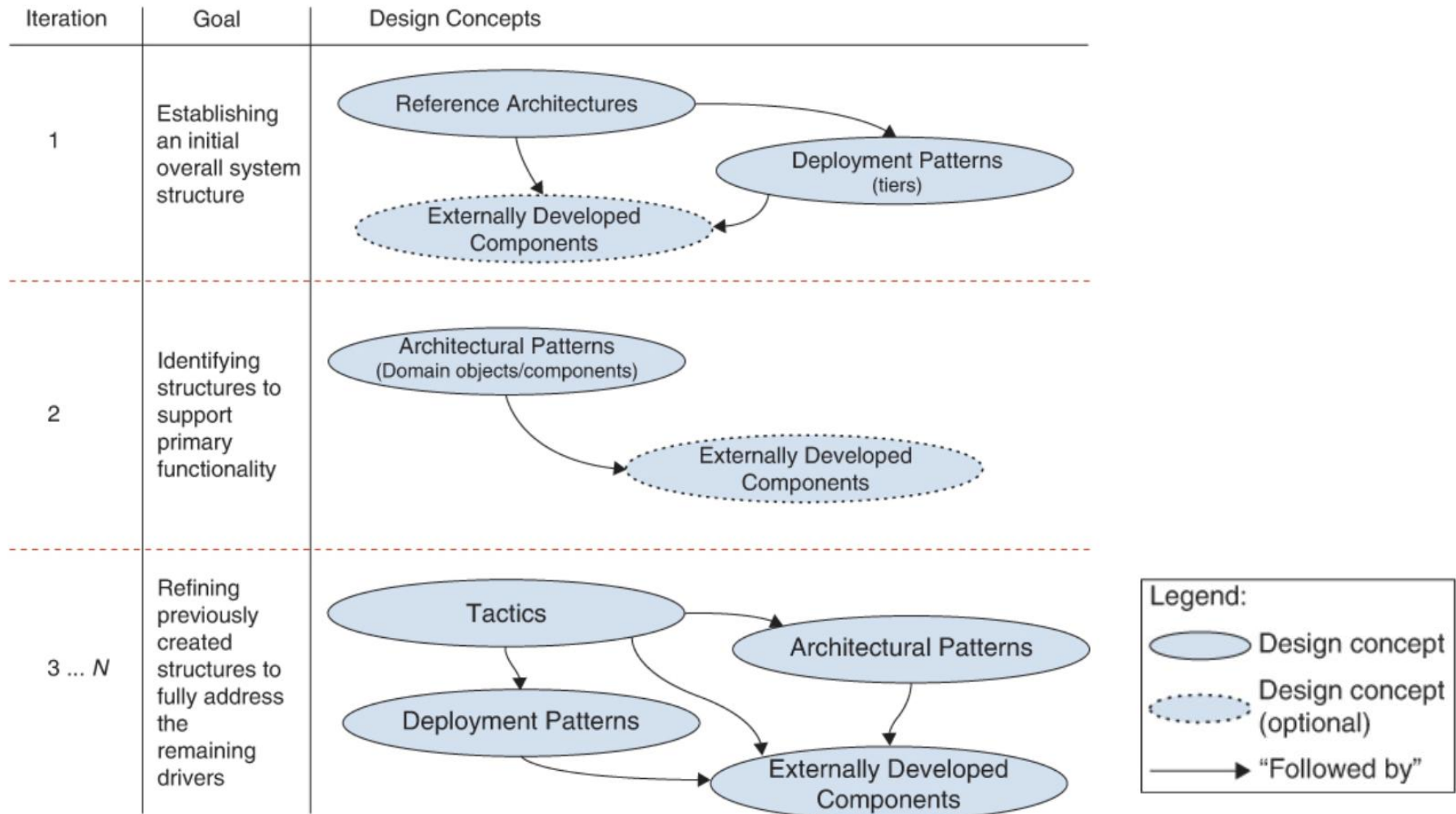
Externally developed components

- These are reusable code solutions
 - E.g. Middleware, frameworks
- A Framework is a reusable software element that provides generic functionality, addressing recurring concerns across a range of applications.
 - Examples for Java:

Concern	Framework	Usage
Local user interface	Swing	Inheritance
Web UI	Java Server Faces (JSF)	XML, Annotations
Component connection	Spring	XML, Annotations
Security (authentication, auth)	Spring-Security	XML, Annotations
OO – Relational Mapping	Hibernate	XML, annotations

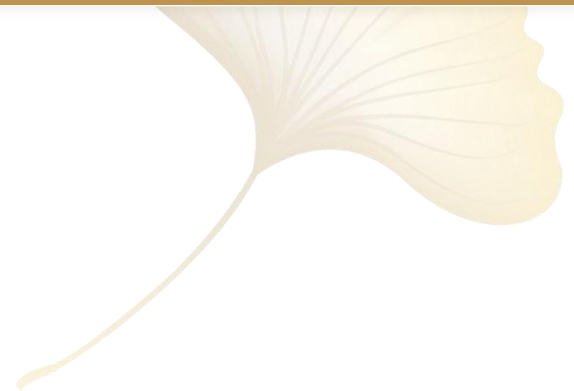
Selection roadmap

Design concept selection roadmap for greenfield systems





Attribute–Driven Design Method (ADD 3.0)



Architecture design methods

- There exist several architecture development methods
 - Viewpoints and Perspectives
 - Microsoft
 - Process of Software Architecting
 - ACDM
 - RUP
 - ADD
- Most of them cover the whole architecture lifecycle and provide few details on how to perform the design activity

Why is a design method necessary?

Ensuring Drivers Are Satisfied Through Methodical Design

- **Architecture design is notoriously difficult to master**
 - Many aspects need to be considered when making design decisions
 - It requires extensive knowledge of the domain and existing solutions
- **Design can (and should) be performed in a systematic way**
 - To ensure that decisions are made with respect to the drivers.
 - To ensure that decisions are recorded and justified and to make the architect accountable for them
 - To provide guidance to less experienced people
- Otherwise, architecture design may end up being seen a mystic activity performed by gurus.



Why is a design method necessary?

Ensuring Drivers Are Satisfied Through Methodical Design

- **A design method**
 - Combing the design process with the elements, relationships, and attributes to construct the desired architecture
- **Introduction to Attribute - Driven Design (ADD 3.0)**
 - Creating high - quality, predictable, and reusable software architectures through a systematic approach
- At the end of the class, participants should be able to understand:
 - The different types of architectural drivers
 - What are design concepts and the decisions regarding their selection
 - How an architecture is designed iteratively using the ADD method

A Brief History of ADD

Tracing Its Development in Architecture Design



January 2000

ADD 1.0

- The **first** design method to focus specifically on quality attributes, architectural structures, and views
- Lack of **specific implementation guidance**.



November 2006

ADD 2.0

- Linking quality attributes to design choices through incorporating a **combination of patterns and tactics**.
- Limitations about **inputs, elements decomposition in each design iteration, drivers, design concepts, and the Initial documentation and analysis**.



2016

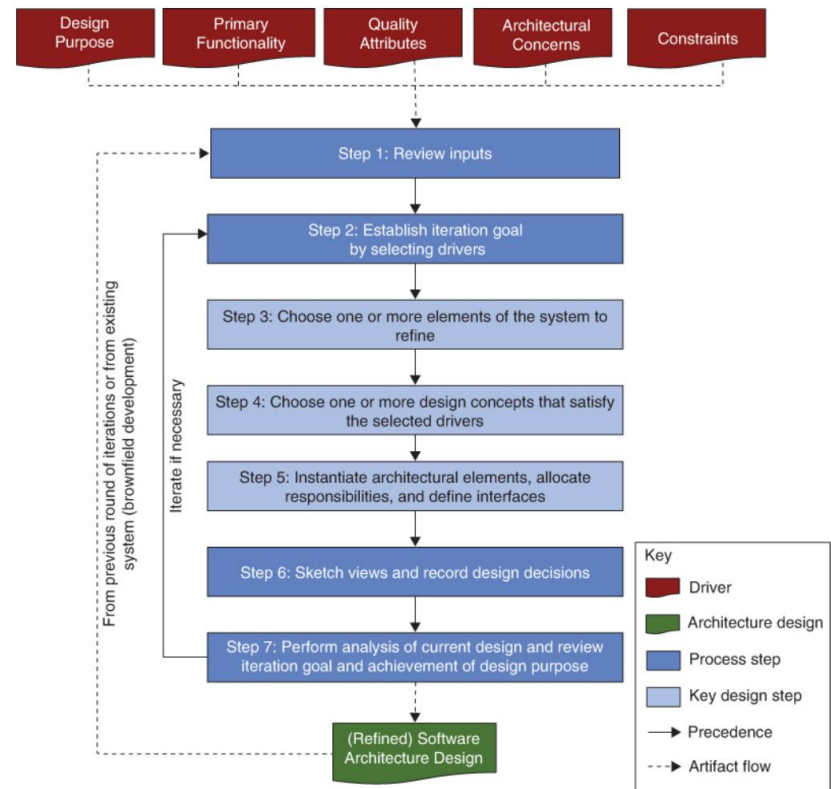
ADD 3.0

- **application frameworks**
- starts with **drivers**, to design **decisions**, to the available **implementation options**
- **quick design iterations** in which a small number of design decisions
- promotes the (re)use of **reference architectures** and promotes the adoption of a **“design concepts catalog”**

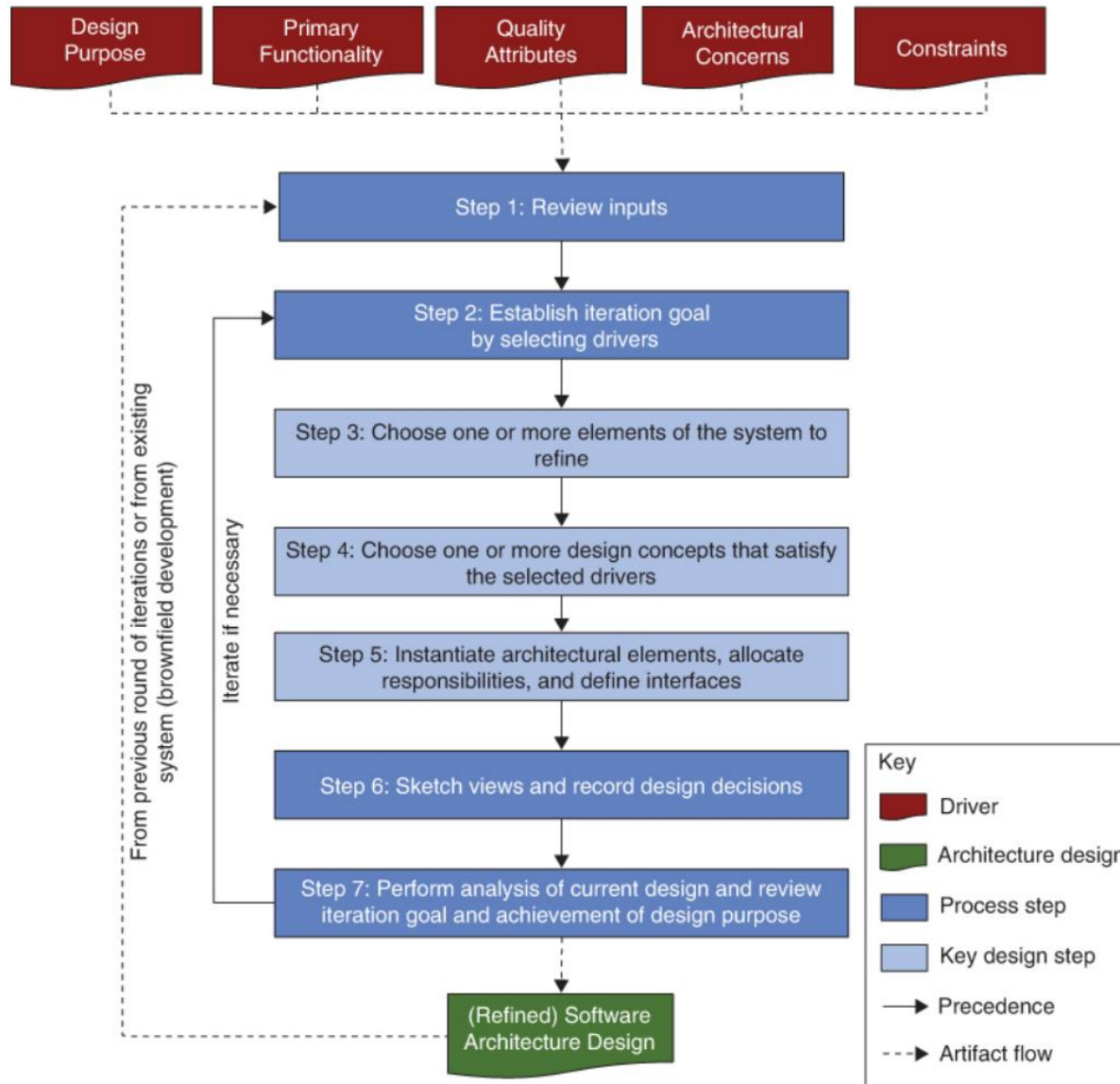
Attribute-Driven Design 3.0 (ADD 3.0)

Modern, Iterative, and Time-Tested

- **Iterative Design Approach:** ADD supports iterative rounds within project increments like sprints, enabling structured design through micro-iterations.
- **Stable and Industry-Proven:** ADD 3.0 remains unchanged despite shifts in development practices, proving its general applicability.
- **Adaptability and Consistency:** Applicable to modern techniques like DevOps and cloud-based architectures without altering the method's core steps.



Attribute-Driven Design 3.0 (ADD 3.0)

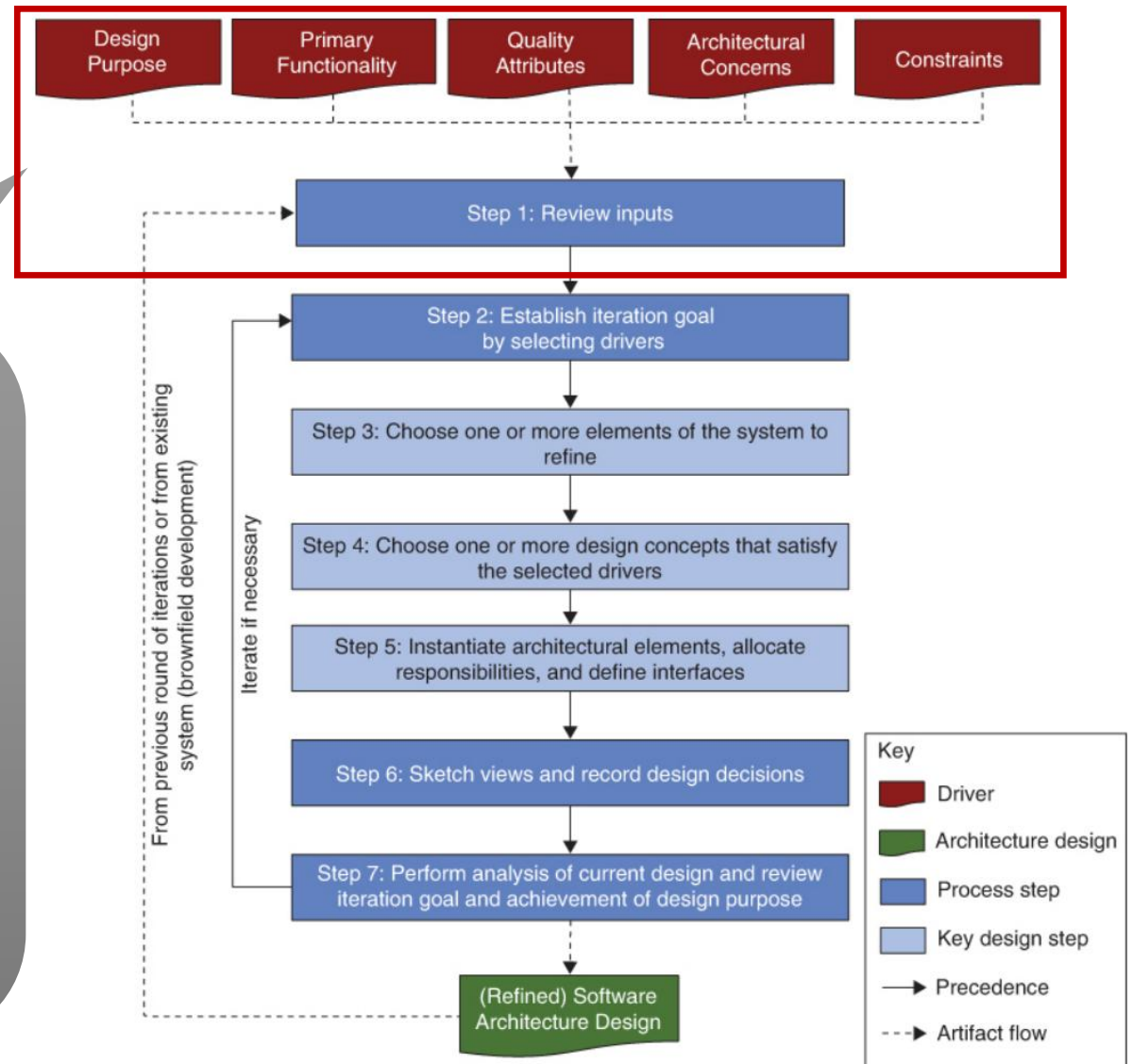


Attribute-Driven Design 3.0 (ADD 3.0)

Step 1: Review Inputs

1. Before starting a design round, you need to make sure that the inputs to the design process are available and correct.

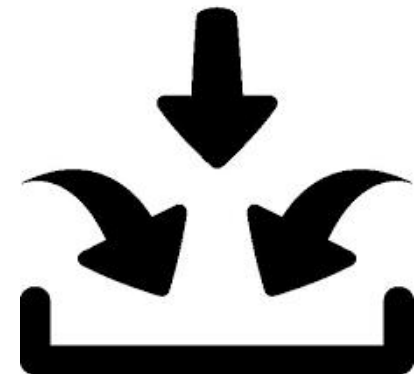
- Design Purpose
- Primary functionality
- Quality attribute scenarios
- Architectural constraints
- Concerns
- Existing architecture design



Attribute-Driven Design 3.0 (ADD 3.0)

Step 1: Review Inputs

- The **primary functionality and quality attribute scenarios** need have been **prioritized**, ideally by your most important project stakeholders.
 - If not, employ various techniques to elicit and prioritize them, e.g., **the Quality Attribute Workshop** and **the Utility Tree**.
- Get the drivers and their priority right (Avoid “garbage in, garbage out”)
 - Check whether any **important stakeholders** were **overlooked** in the original requirements elicitation process
 - Check whether any **business conditions** have **changed** since the prioritization was performed.
- **Start** designing...
- Perform the different design **iterations**
 - other quality attribute scenarios
 - use cases
 - architectural concerns
 -

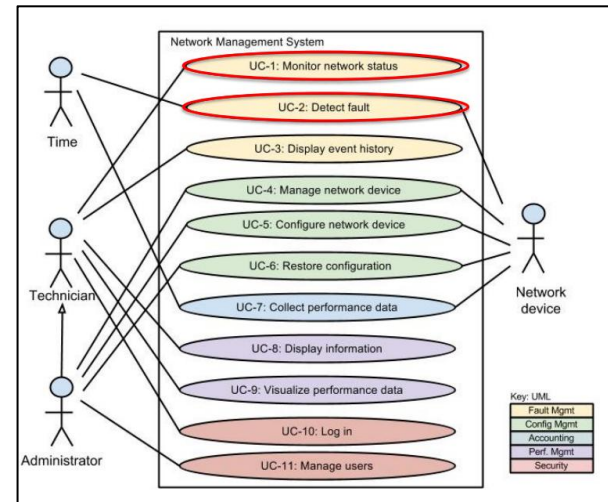


Attribute-Driven Design 3.0 (ADD 3.0)

Step 1: Review Inputs

Example

- Type Of System: Greenfield in mature domain
- Objective: Design in preparation for construction of an increment of the system
- Concerns
 - Structure the system
 - Organization of the codebase
 - Input validation
 - Exception management
 - ...

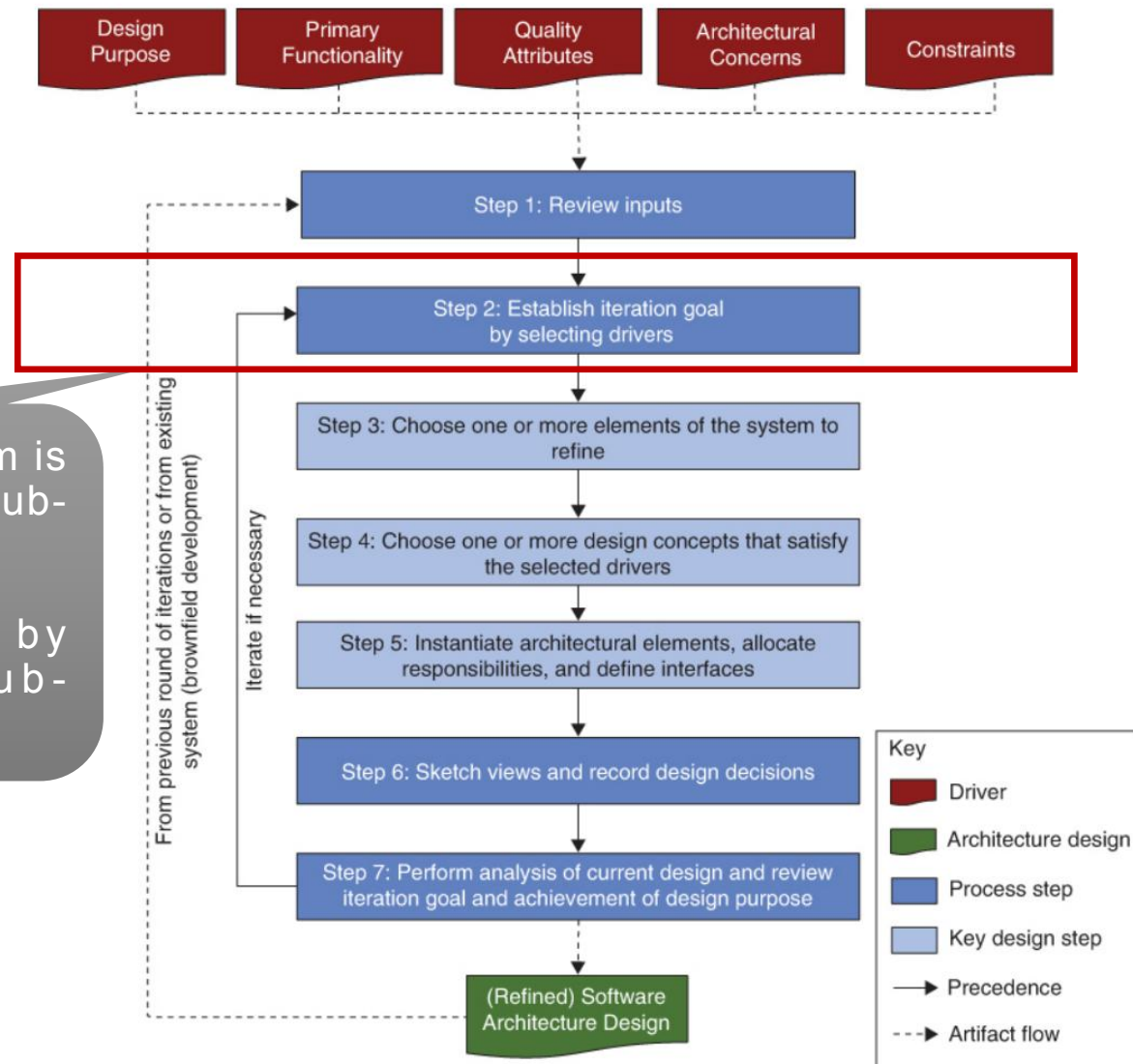


ID	Quality Attribute	Scenario	Associated use case	Priority
QA-1	Performance	Several network devices send traps to the management system at peak load. 100% of the traps are successfully processed and stored.	Detect network device fault (UC-2)	H, H
QA-2	Modifiability	A new network device management protocol is introduced to the system as part of an update. The protocol is added successfully without any changes to the core components of the system.	Configure network device (UC-5)	M, M
QA-3	Availability	A failure occurs in the management system during operation. The management system resumes operation in less than 30 seconds.	All	H, H
QA-4	Performance	The management system collects performance data from a network device during peak load. The management system collects all performance data within 5 minutes to ensure no loss of data.	Collect performance data (UC-7)	H, H
QA-5	Performance, Usability	A user displays the event history of a particular network device during normal operation. The list of events from the last 24 hours is displayed within 1 second.	Display Event history (UC-3)	H, M
QA-6	Security	A user performs a change in the system during normal operation. It is possible to know who performed the operation and when it was performed 100% of the time.	All	H, M

ID	Constraint
CON-1	A minimum of 50 simultaneous users must be supported.
CON-2	User workstations use the following operating systems: Windows, OSX, and Linux.
CON-3	An existing relational database server must be used.
CON-4	Network connection between users workstations and the server is unreliable
CON-5	Future support for mobile clients
CON-6	A minimum of 600 time servers must be supported (Initial deployment 100 time servers, 100 more / year during 5 years)
CON-7	Each time server sends, on average, 10 traps/hour.
CON-8	Performance data needs to be collected in intervals of no more than 5 minutes as higher intervals result in time servers discarding data.
CON-9	Events from the last 30 days must be stored
CON-10	The development team is familiar with Java technologies

Attribute-Driven Design 3.0 (ADD 3.0)

Step 2: Establish the Iteration Goal by Selecting Drivers



2. The design problem is divided into several sub-problems.

An iteration starts by deciding which sub-problem to address.

Attribute-Driven Design 3.0 (ADD 3.0)

Step 2: Establish the Iteration Goal by Selecting Drivers

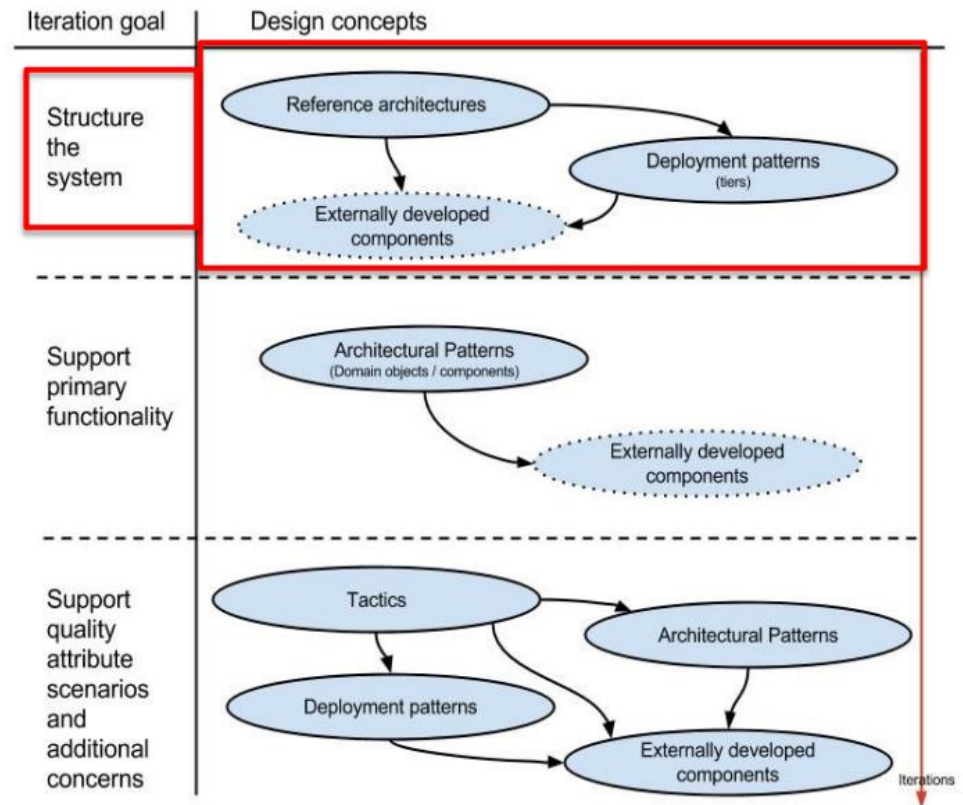
- A design round generally takes the form of a series of design iterations, where each iteration focuses on achieving a particular goal.
- Such a goal typically involves designing to satisfy a subset of the drivers.
 - Create structures from elements that will support a particular performance scenario
 - Enable a use case to be achieved
- Establish a “right-sized” iteration goal:
 - may be to address a single important driver,
 - the architectural concern of decomposing the system
 - addressing a challenging quality attribute scenario.
 - may be to make decisions to satisfy a set of similar drivers,
 - a family of use cases or related quality attribute scenarios
 - may be to make decisions to satisfy a set of related drivers
 - a user story and an associated quality attribute scenario.



Attribute-Driven Design 3.0 (ADD 3.0)

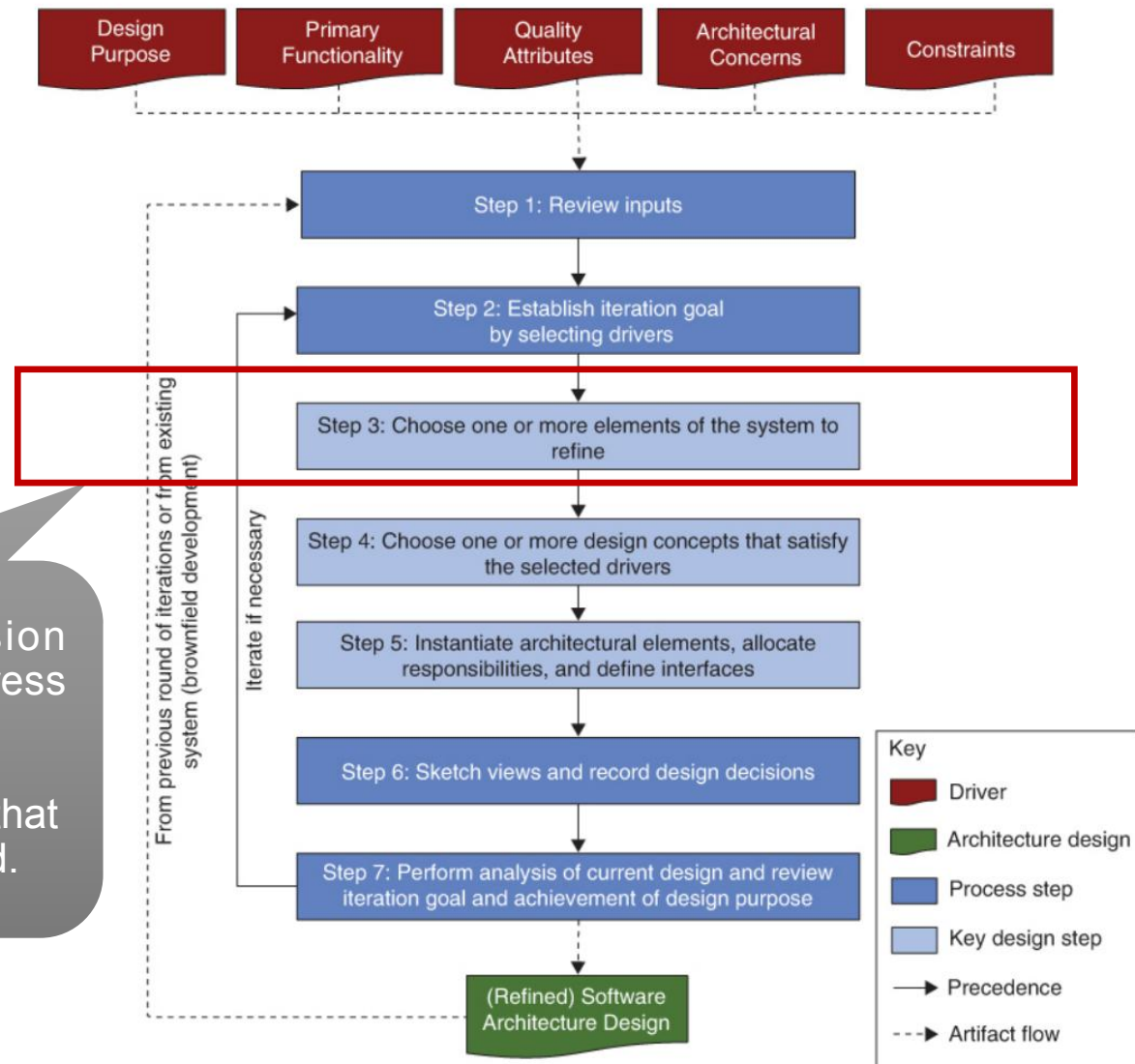
Step 2: Establish the Iteration Goal by Selecting Drivers

- **ADD : Iteration 1**
- Iteration goal and inputs
 - Iteration goal
 - Create an overall system structure
 - Inputs to be considered
 - All



Attribute-Driven Design 3.0 (ADD 3.0)

Step 3: Choose One or More Elements of the System to Refine



3. One of three decision types are made to address the sub-problem:

- Selection of the parts that need to be decomposed.

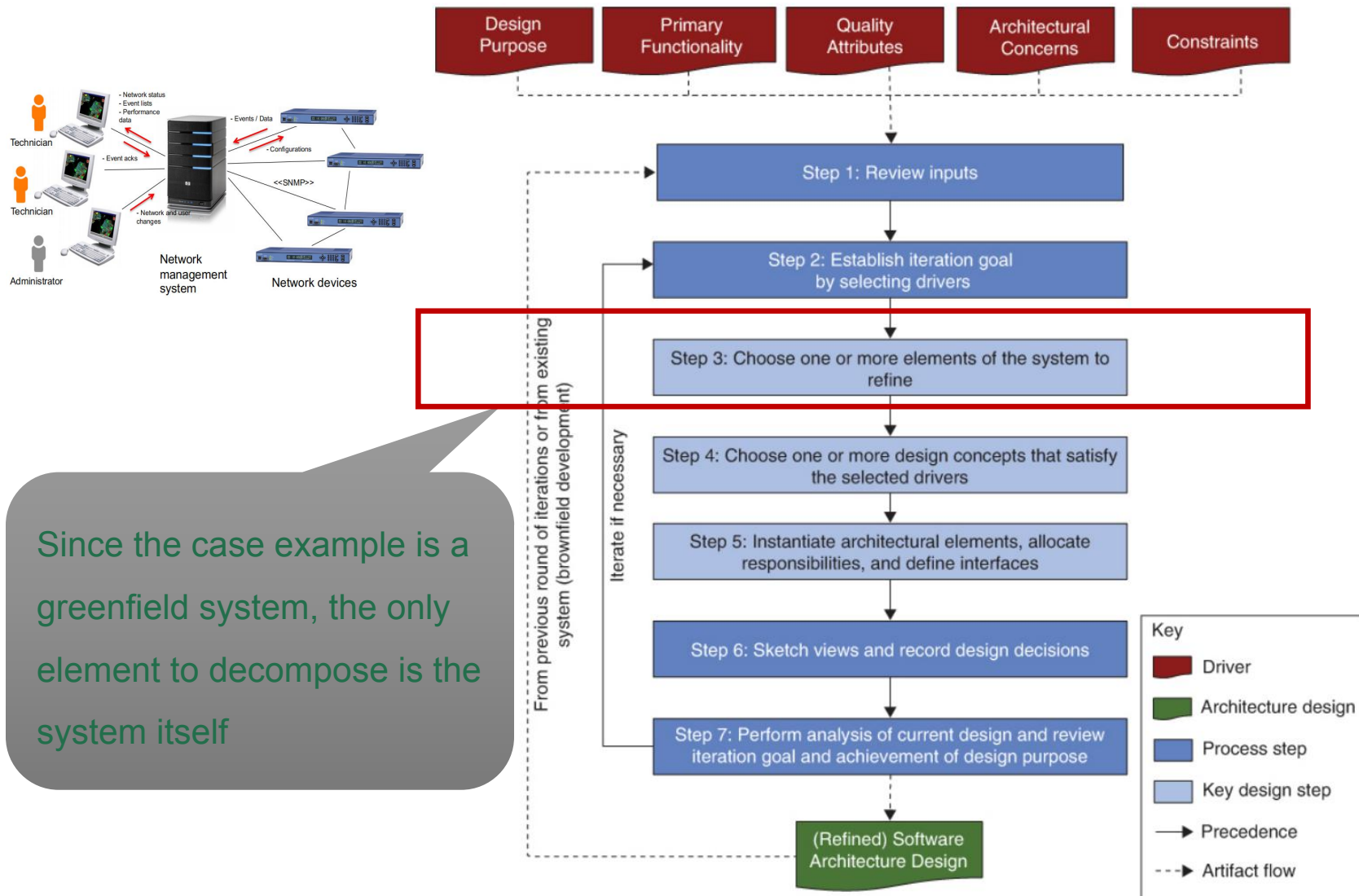
Attribute-Driven Design 3.0 (ADD 3.0)

Step 3: Choose One or More Elements of the System to Refine

- Refinement can mean:
 - decomposition into finergrained elements (top-down approach),
 - combination of elements into coarser-grained (bottom-up approach)
 - improvement of previously identified elements.
- For greenfield development, you can start by:
 - establishing the system context
 - selecting the only available element (the system itself) for refinement by decomposition
- For existing systems or for later design iterations in greenfield systems:
 - refine elements that were identified in prior iterations
 - need a good understanding of the elements of the system
- Step 2 precedes step 3. In some cases the order can be reversed.

Attribute-Driven Design 3.0 (ADD 3.0)

Step 3: Choose One or More Elements of the System to Refine



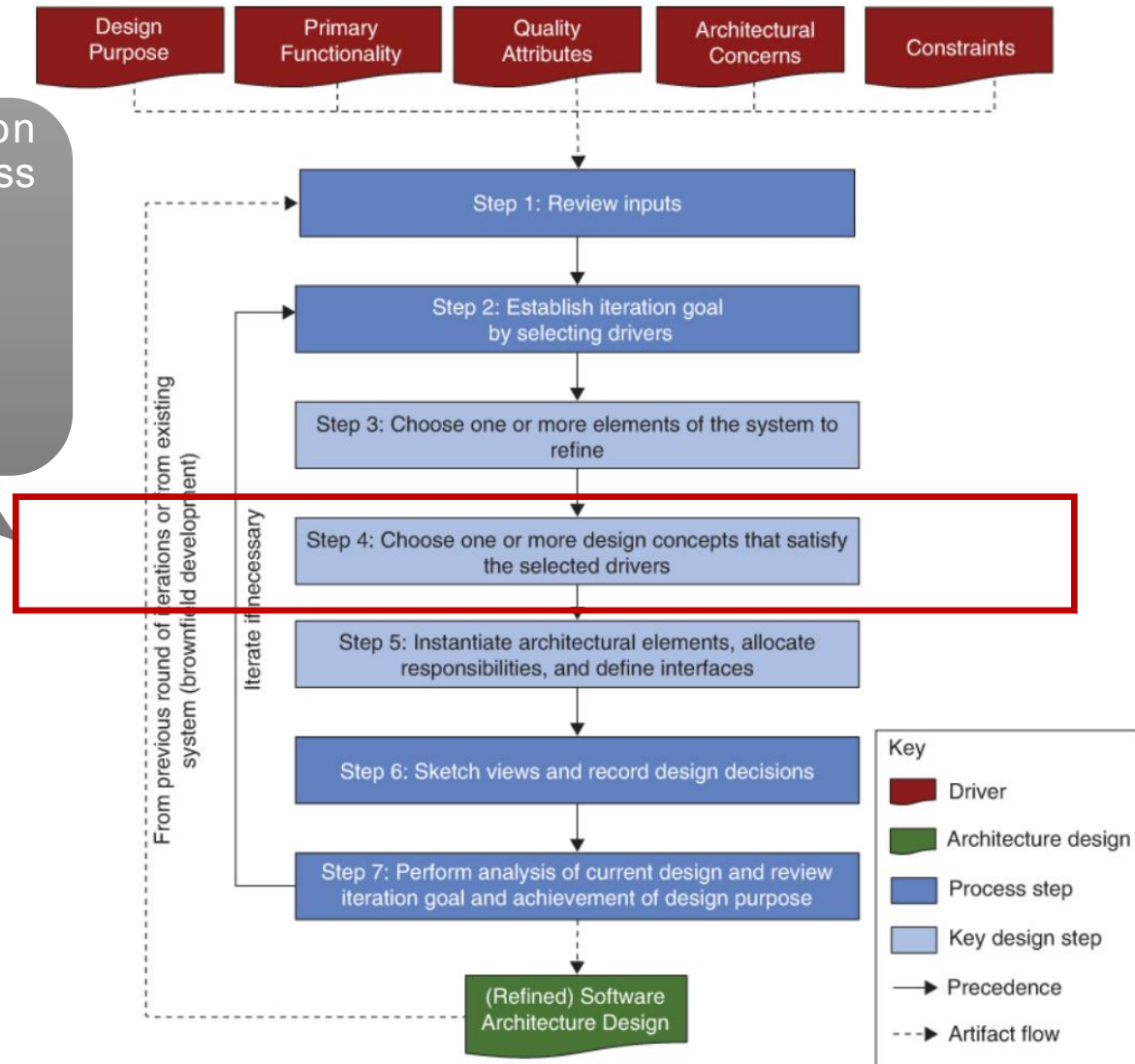
Since the case example is a greenfield system, the only element to decompose is the system itself

Attribute-Driven Design 3.0 (ADD 3.0)

Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

4. One of three decision types are made to address the sub problem:

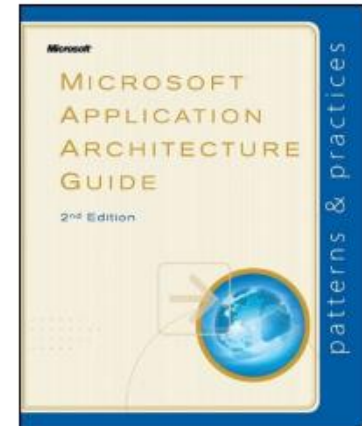
- Identification and selection of existing solutions that support the decomposition.



Attribute-Driven Design 3.0 (ADD 3.0)

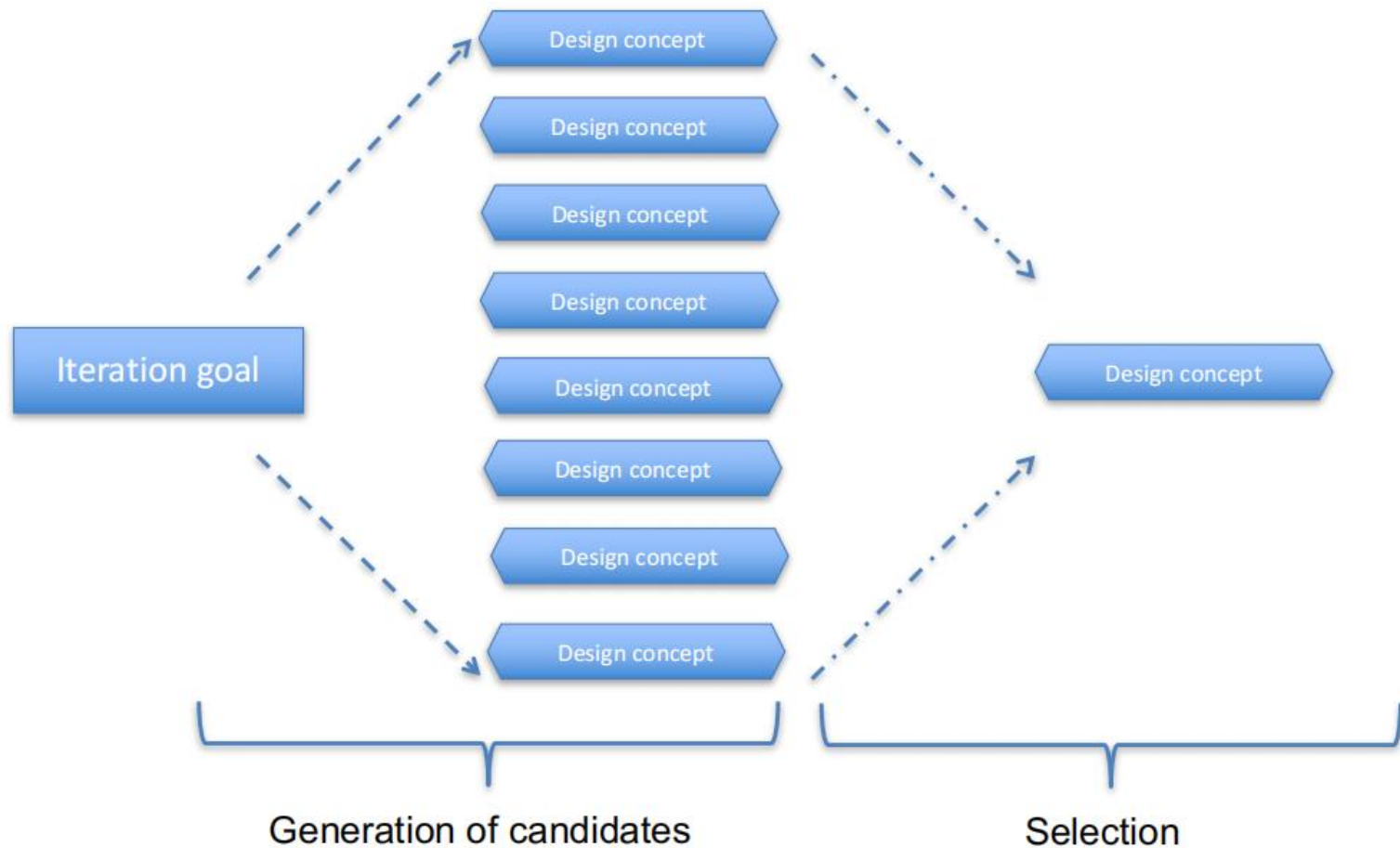
Step 4.1: Identify Design Concepts

- Reference architecture alternatives
 - Mobile applications
 - Rich client applications
 - Rich internet applications
 - Service Applications
 - Web applications
- Distributed deployment patterns alternatives
 - 2 tier
 - 3 tier
 - 4 tier



Attribute-Driven Design 3.0 (ADD 3.0)

Step 4.2: Selection of Design Concepts



Attribute-Driven Design 3.0 (ADD 3.0)

Step 4.2: Selection of Design Concepts

- **Basic Selection Method**

- Create a **pros-cons** table listing advantages and disadvantages of each alternative design concept to support selection based on drivers.

Name of Alternative	Pros	Cons
Monolithic application	Easier to manage and deploy.	Updates require redeployment of the whole application. Horizontal scaling requires creating a replica of the entire application.
Microservices-based application	Services can be replicated and updated independently.	More complex management.

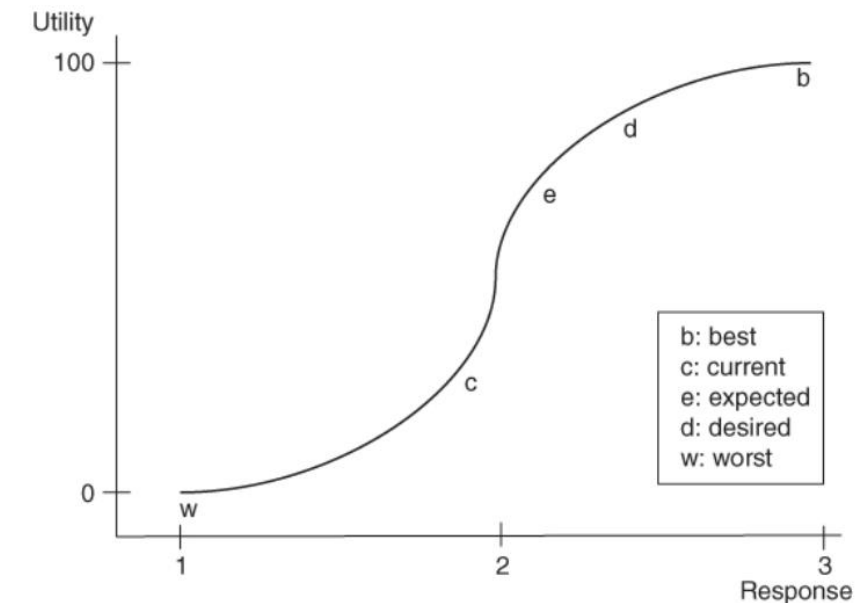
Example of a Table to Support the Selection of Alternatives

Attribute-Driven Design 3.0 (ADD 3.0)

Step 4.2: Selection of Design Concepts

- **In - depth Analysis Approaches**

- **CBAM**: Quantitative method using utility - response curves, cost - benefit analysis, and ROI to select architectural strategies.
- **SWOT**: Evaluate strengths, weaknesses, opportunities, and threats of alternatives.



Utility-response curve

Attribute-Driven Design 3.0 (ADD 3.0)

Step 4.2: Selection of Design Concepts

- **Prototype - Based Selection**

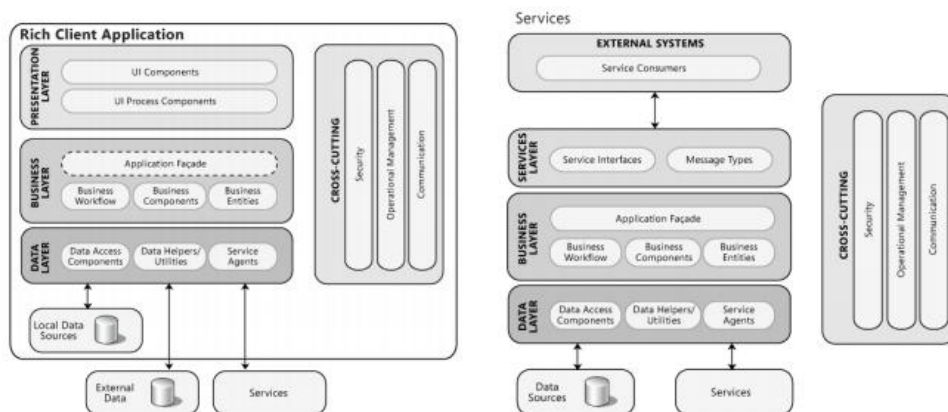
- When analysis fails, create throwaway prototypes.
- Creation of prototypes can be **costly** compared to analysis (the cost ratio is between 10 and 5 to 1).
- **Certain scenarios** strongly motivate it, e.g., consider prototyping if project involves **emerging/new tech** for the company, or **uncertain tech integration**.
- Aspects that you should consider when deciding whether creating a prototypes:

- Does the project incorporate emerging technologies?
- Is the technology new to the company?
- Are there certain drivers, particularly quality attributes, whose satisfaction using the selected technology presents risks (i.e., it is not understood whether they can be satisfied)?
- Is there a lack of trusted information, internal or external, that provides some degree of certainty that the selected technology will be useful to satisfy the project drivers?
- Are there configuration options associated with the technology that need to be tested or understood?
- Is it unclear whether the selected technology can be integrated with other technologies that are used in the project?

Attribute-Driven Design 3.0 (ADD 3.0)

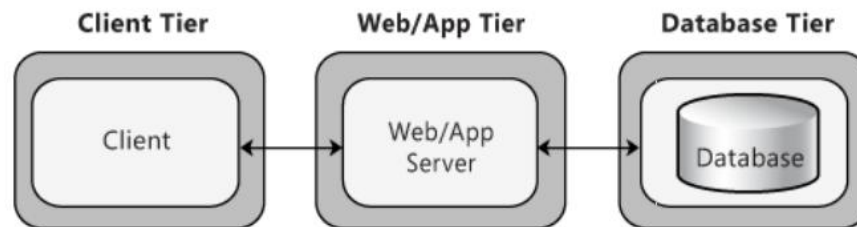
Step 4.2: Selection of Design Concepts

- Two reference architectures are chosen



Design decision	Rationale
Rich client application on the client side	- Supports rich user interface - Portability (CON-2)
Service application on the server side	- Support mobile clients in the future (CON-5)
3 Tier application	- Existing database server (CON-3)
...	...

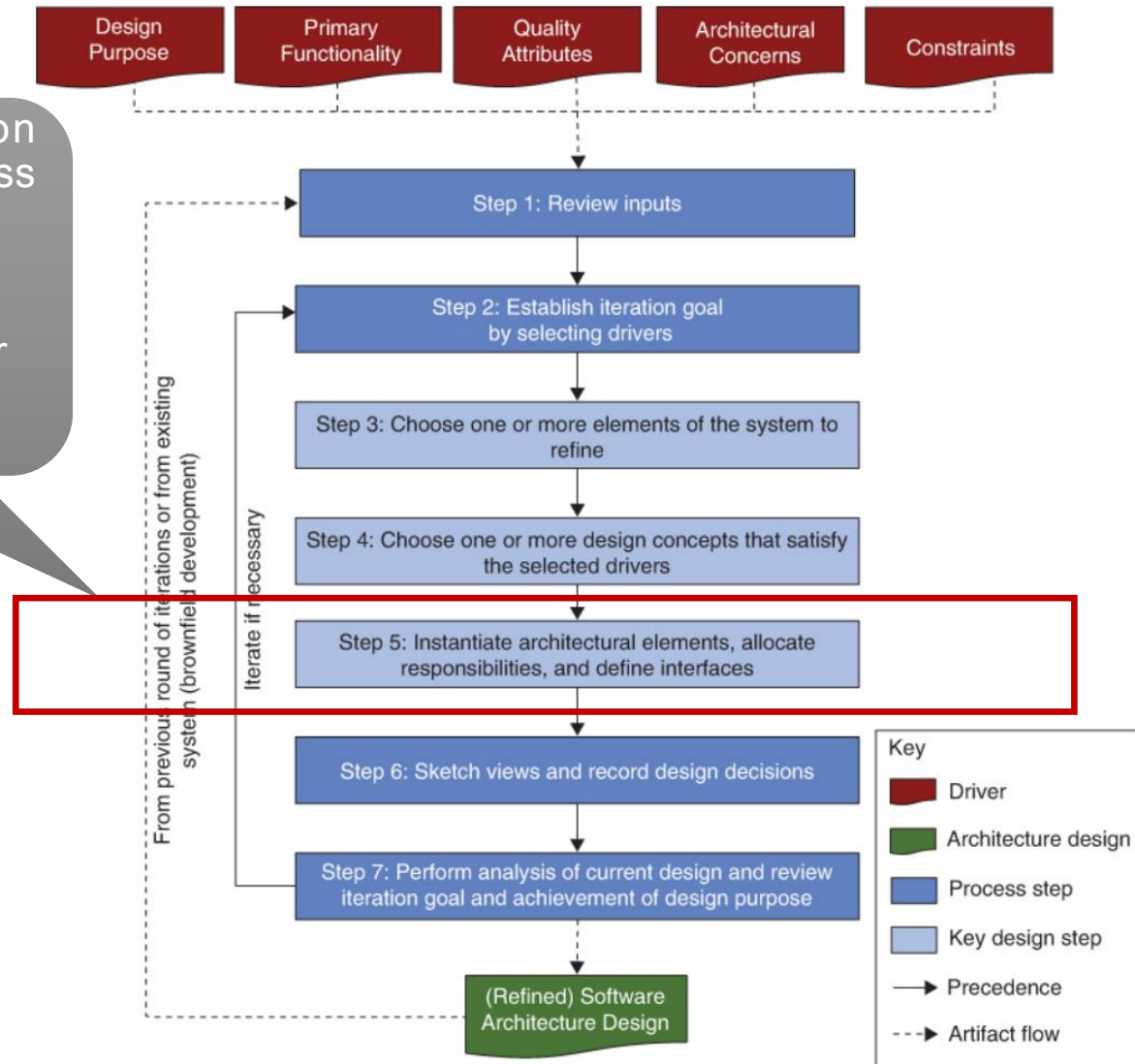
- Distributed deployment patterns alternatives
 - 3 tier



Attribute-Driven Design 3.0 (ADD 3.0)

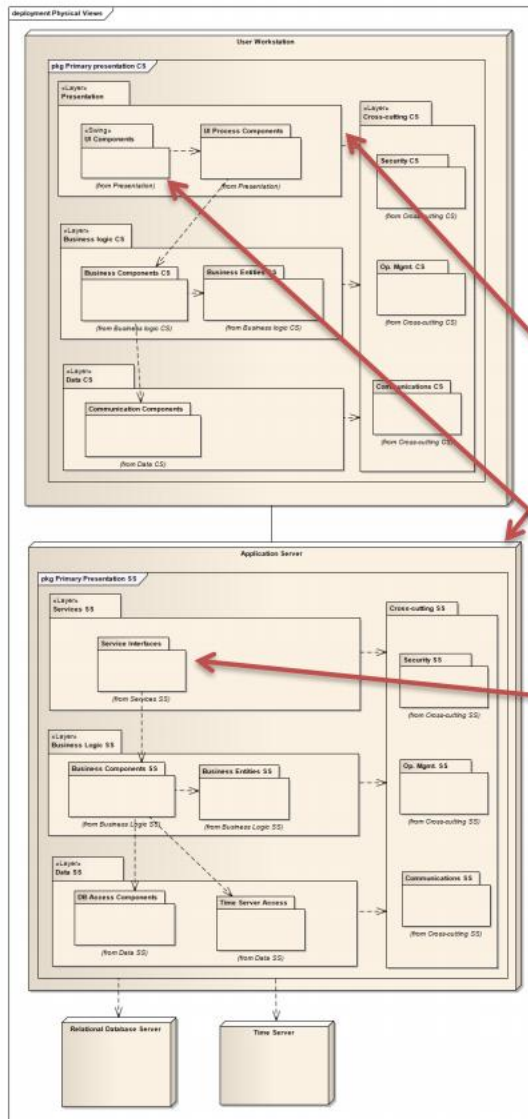
Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

5. One of three decision types are made to address the sub problem:
- Creation of elements from the existing solution and establishment of their responsibilities and interfaces.



Attribute-Driven Design 3.0 (ADD 3.0)

Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces



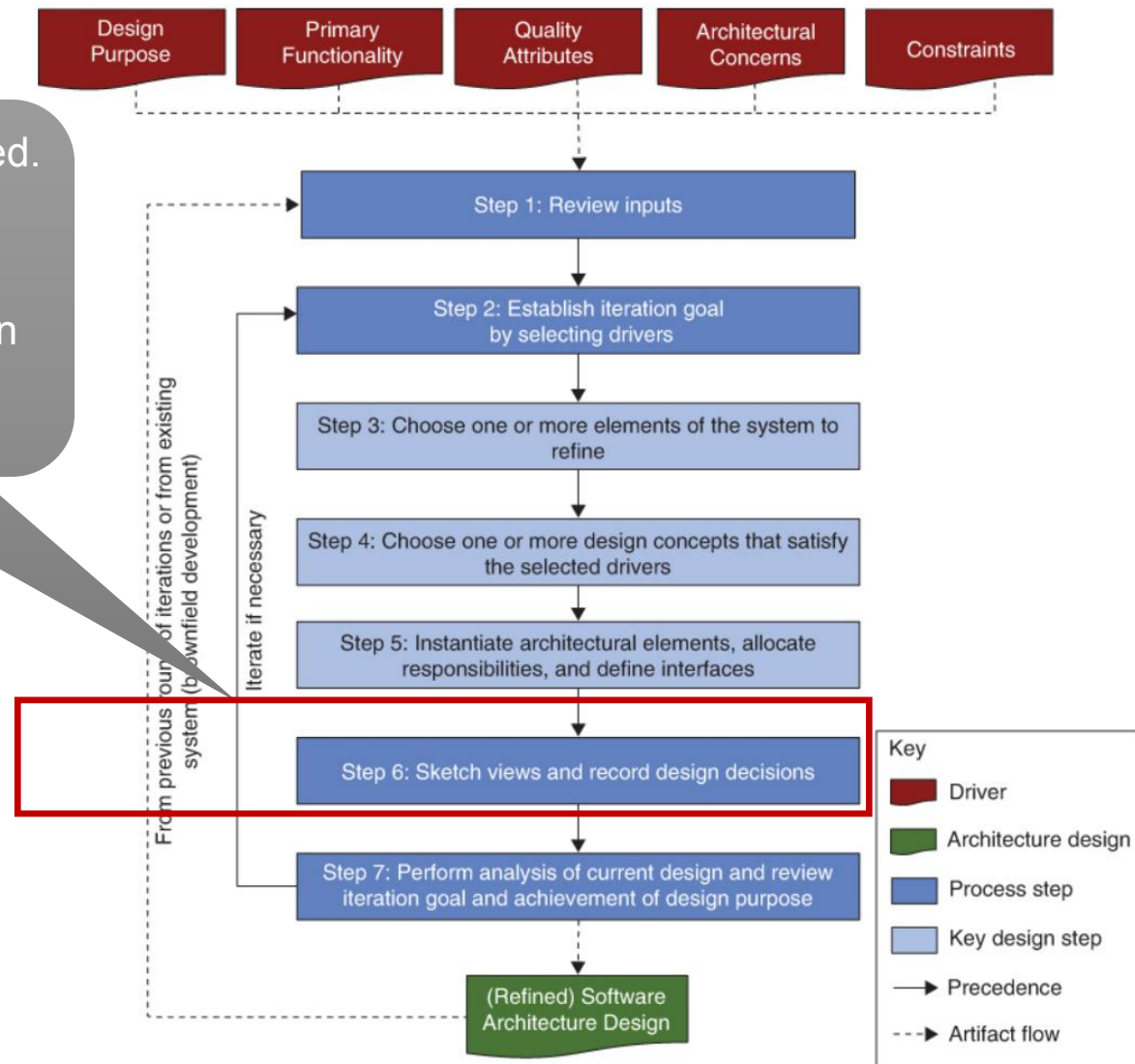
Element	Responsibility	Properties
Application Server	- Hosts the elements of the service reference architecture - Connects to network devices - Connects to database server	OS = Centos
Presentation layer (Client side)	This layer contains components which control user interaction and use case control flow.	
UI Components (Client side)	These are components which render the user interface and receive user interaction.	
Service Interfaces	These is a group of components that expose services that are consumed by the clients	
...	...	

Attribute-Driven Design 3.0 (ADD 3.0)

Step 6: Sketch Views and Record Design Decisions

6. The “blueprint” is refined. This may be done in parallel with step 5.

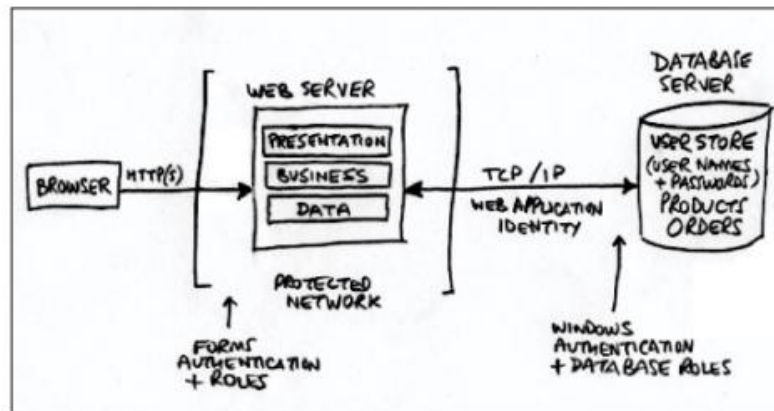
Note: This is not full blown documentation but rather sketches.



Attribute-Driven Design 3.0 (ADD 3.0)

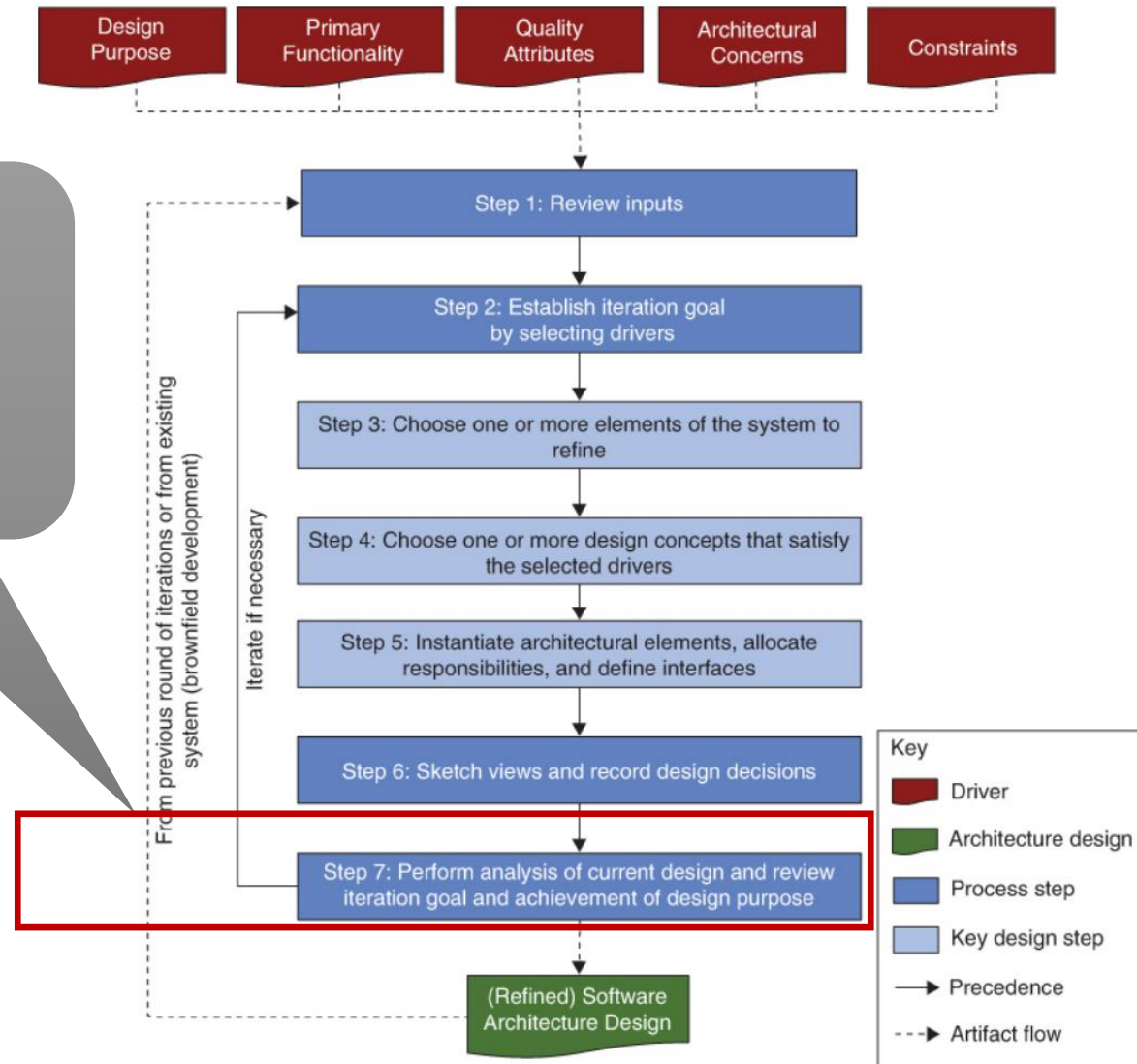
Step 6: Sketch Views and Record Design Decisions

- As you instantiate design concepts you will typically **create sketches**. These are **initial documentation** for your architecture.
 - capture them and flesh them out later
 - if you use informal notation, be consistent
 - develop a discipline of writing down the **responsibilities** that you **allocate to elements** and the **relevant design decisions** that you have made
- **Recording during design** ensures you won't have to **remember** things later...



Attribute-Driven Design 3.0 (ADD 3.0)

Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

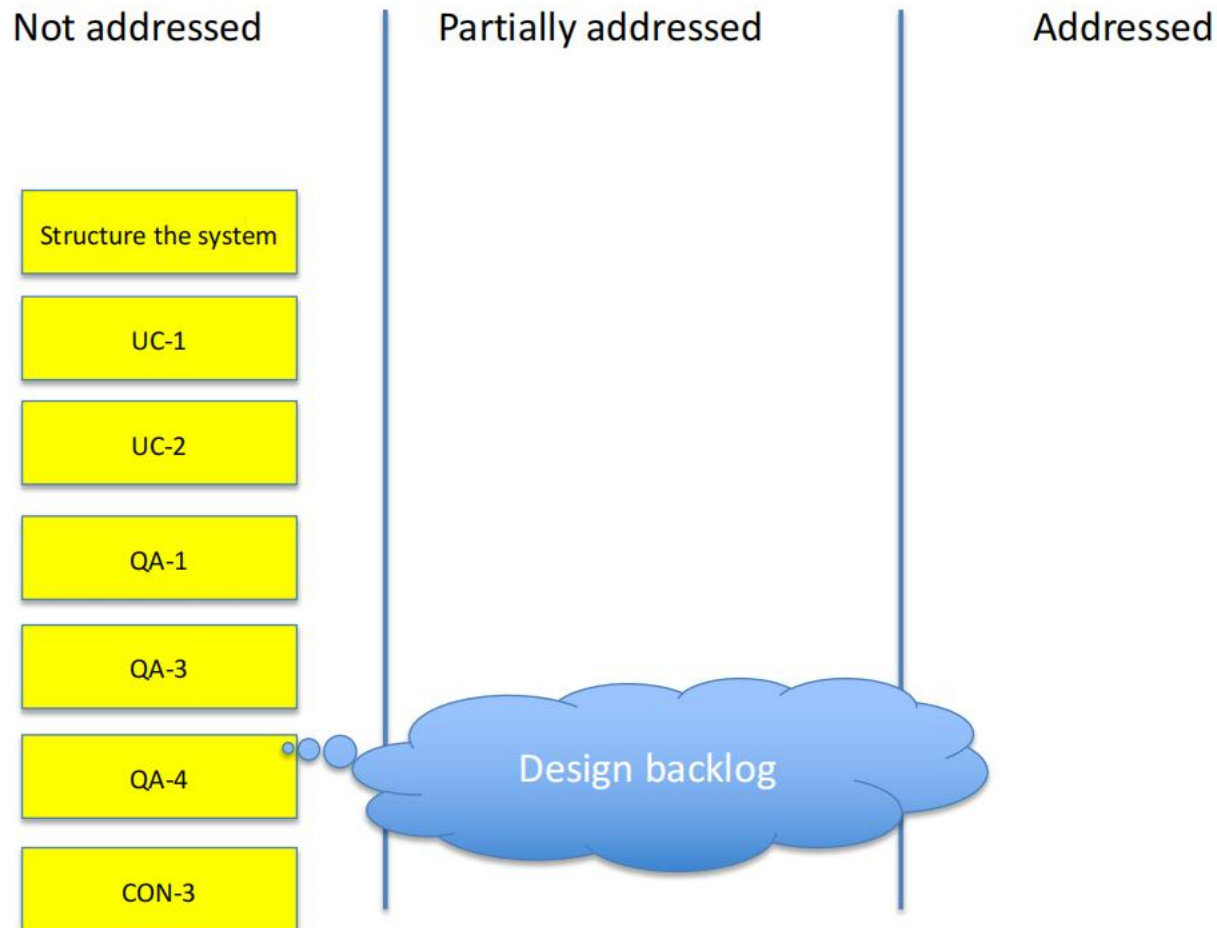


7. Decisions made at this point are analyzed along with the advances in the overall design process in order to decide if more iterations are necessary.

Attribute-Driven Design 3.0 (ADD 3.0)

Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

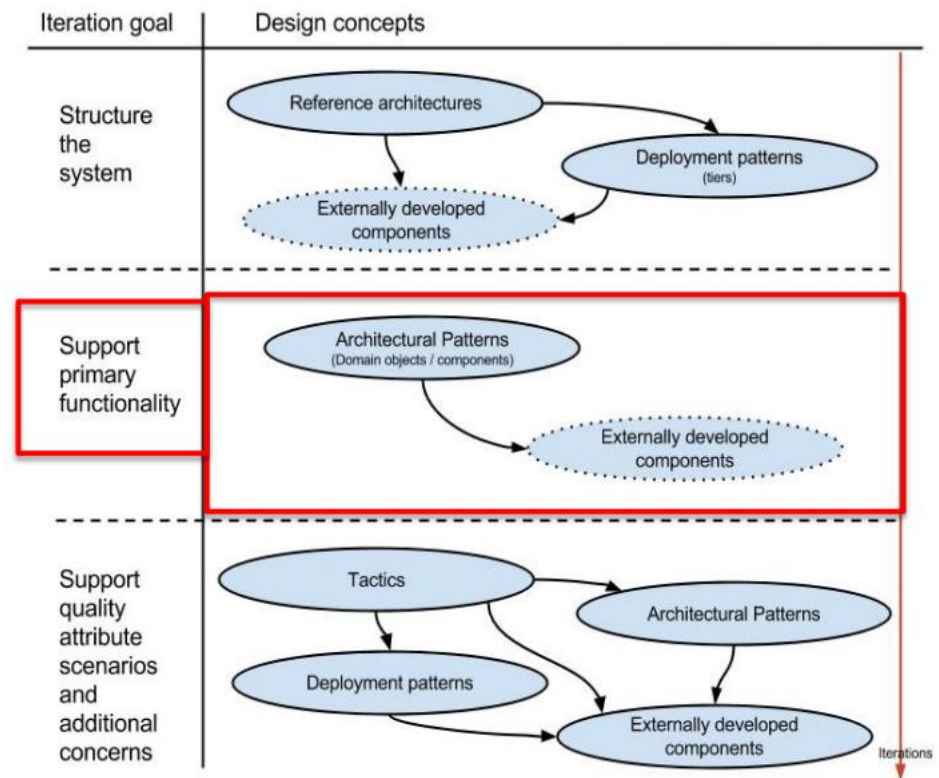
- Design Kanban board



Attribute-Driven Design 3.0 (ADD 3.0)

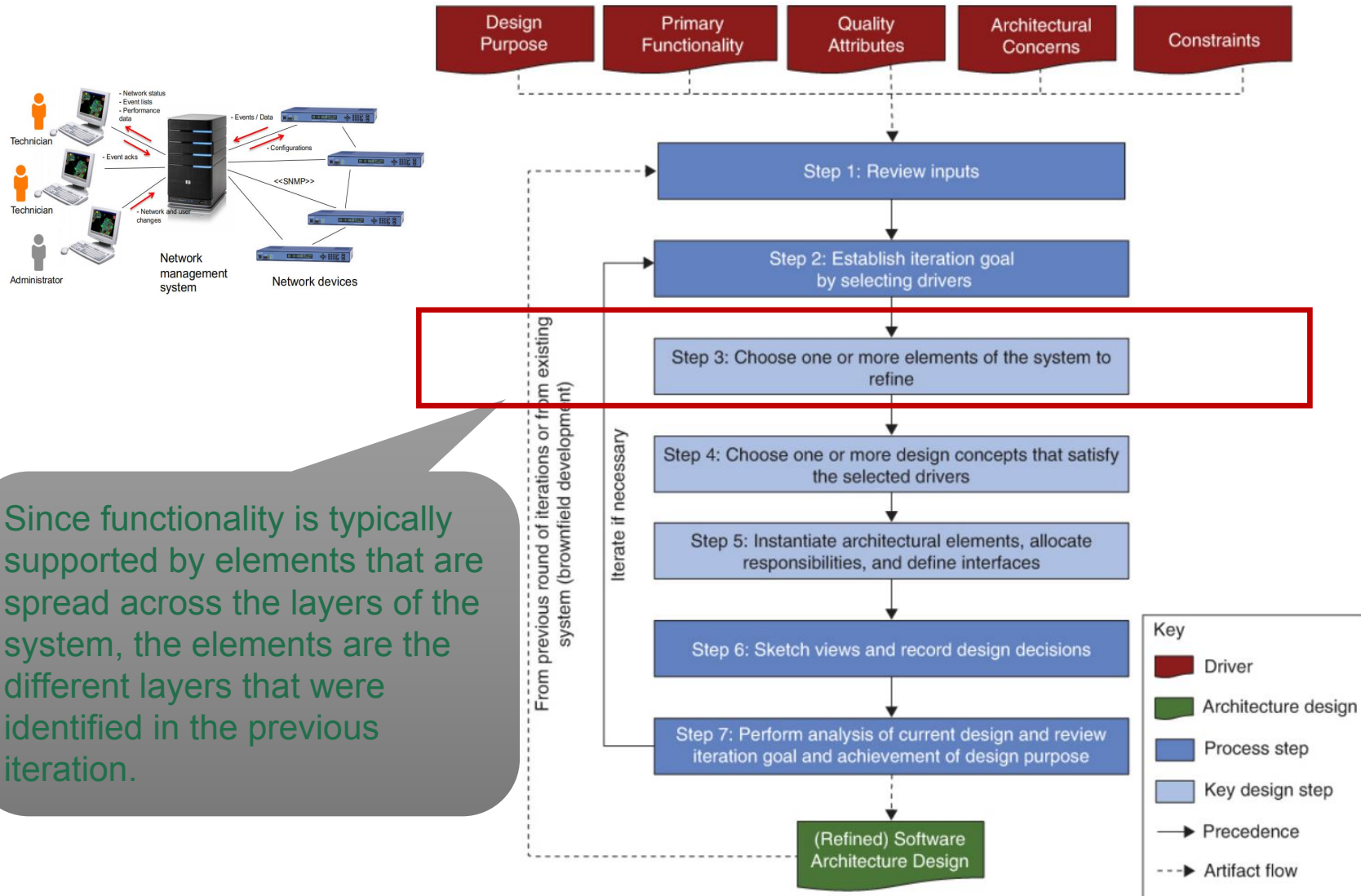
Step 2: Establish the Iteration Goal by Selecting Drivers

- **ADD : Iteration 2**
- Iteration goal
 - Identify the elements that support the primary functionality
- Inputs
 - Primary Use Cases



Attribute-Driven Design 3.0 (ADD 3.0)

Step 3: Choose One or More Elements of the System to Refine

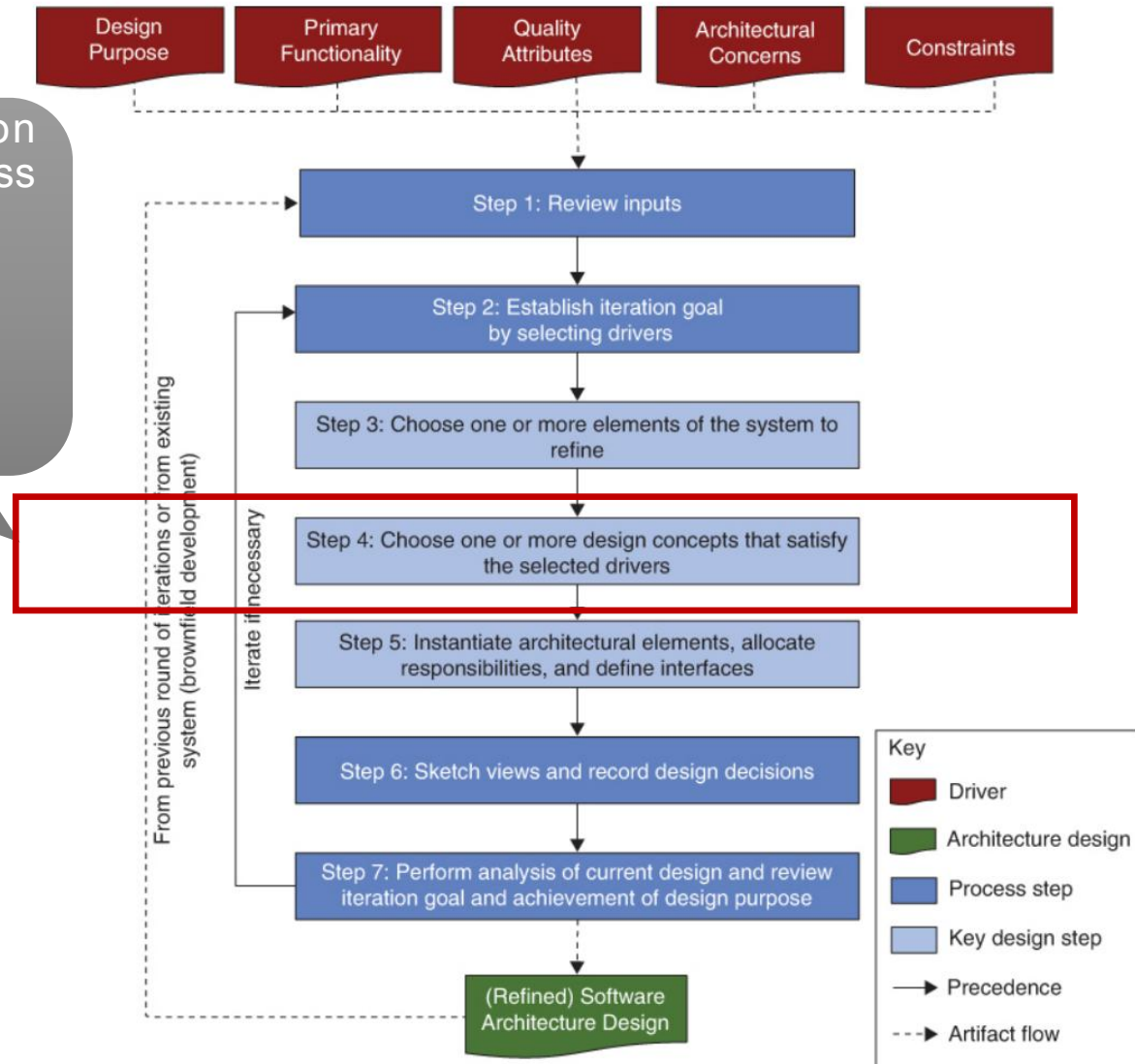


Attribute-Driven Design 3.0 (ADD 3.0)

Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

4. One of three decision types are made to address the sub problem:

- Identification and selection of existing solutions that support the decomposition.

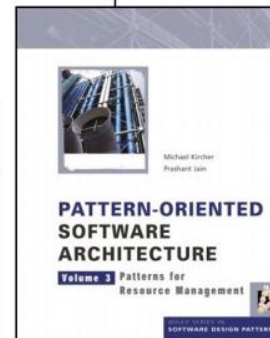


Attribute-Driven Design 3.0 (ADD 3.0)

Step 4: Selection of Design Concepts

- Design concepts
 - Domain objects
 - Externally developed components

Design decision	Rationale
Use domain objects (e.g. components) to decompose layers	Decomposition of the layers facilitates work assignment and associating technologies to components
Use Swing Framework for UI	Developers are familiar with this framework (CON-10)
Use Hibernate for OR Mapping	- CON-10 - New Database (not legacy)



208

From Mud To Structure

Domain Object **

When realizing a DOMAIN MODEL (182), or its technical architecture in terms of LAYERS (185), MODEL-VIEW-CONTROLLER (188), PRESENTATION-ABSTRACTION-CONTROL (191), MICROKERNEL (194), REFLECTION (197), PIPES AND FILTERS (200), SHARED REPOSITORY (202), or BLACKBOARD (205) ...

... a key concern of all design work is to decouple self-contained and coherent application responsibilities from one another.

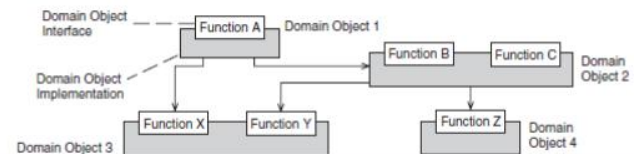


The parts that make up a software system often expose manifold collaboration and containment relationships to one another. However, implementing such interrelated functionality without care can result in a design with a high structural complexity.

Separation of concerns is a key property of well-designed software. The more decoupled are the different parts of a software system, the better they can be developed and evolved independently. The fewer relationships the parts have to one another, the smaller the structural complexity of the software architecture. The looser the parts are coupled, the better they can be deployed in a computer network or composed into larger applications. In other words, a proper partitioning of a software system avoids architectural fragmentation, and developers can better maintain, evolve and reason about it. Yet despite the need for clear separation of concerns, the implementation of and collaboration between different parts in a software system must be effective and efficient for key operational qualities, such as performance, error handling, and security.

Therefore:

Encapsulate each distinct functionality of an application in a self-contained building-block—a domain object.

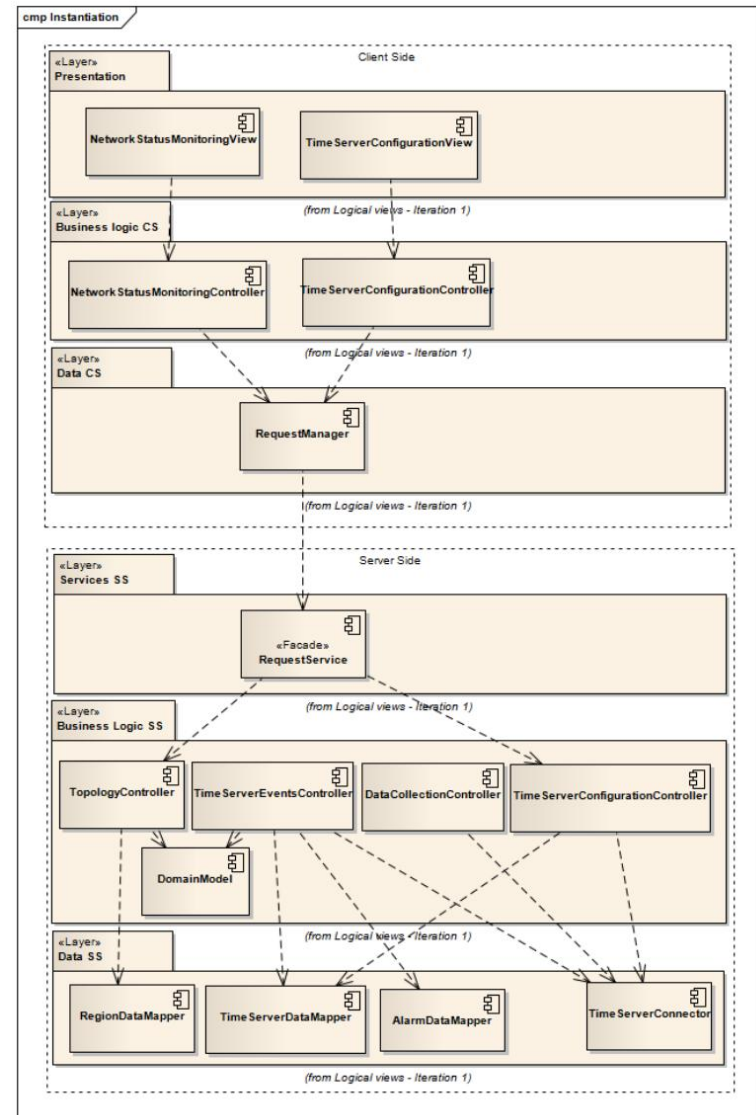


Attribute-Driven Design 3.0 (ADD 3.0)

Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

- Logic View

Element	Responsibility	Properties
NetworkDeviceConnector	Communicate with network devices and isolate the rest of the system from specific protocol	
NetworkDeviceEventController	Process events that are received from the network devices	
Topology Controller	Provides access to the network topology information and changes in it	Type = stateless
Region Data Mapper	Manage persistence of Regions	Framework = Hibernate
NetworkStatusMonitoringView	Display the network topology and events that occur on the devices	Framework = Swing

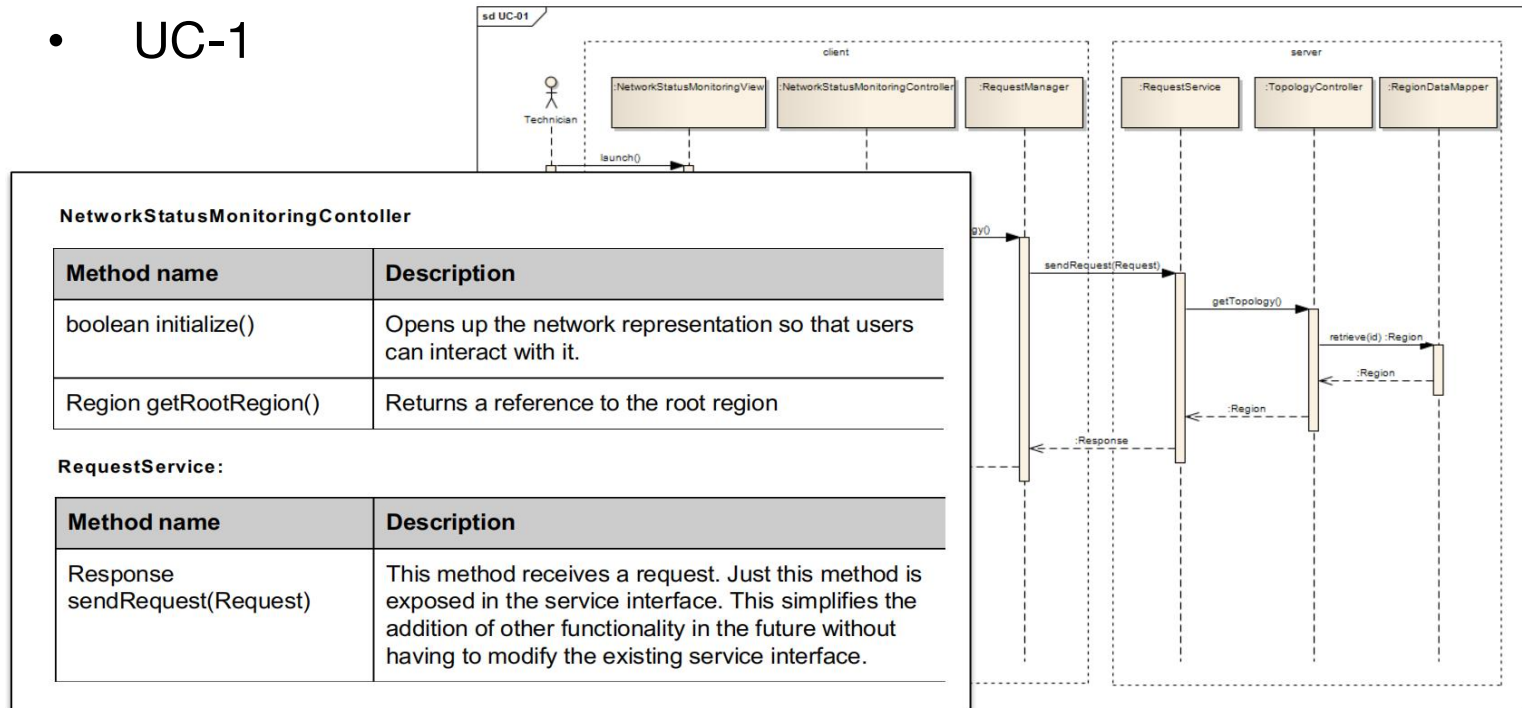


Attribute-Driven Design 3.0 (ADD 3.0)

Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

- Once the elements have been identified, dynamic analysis allows interfaces to be defined

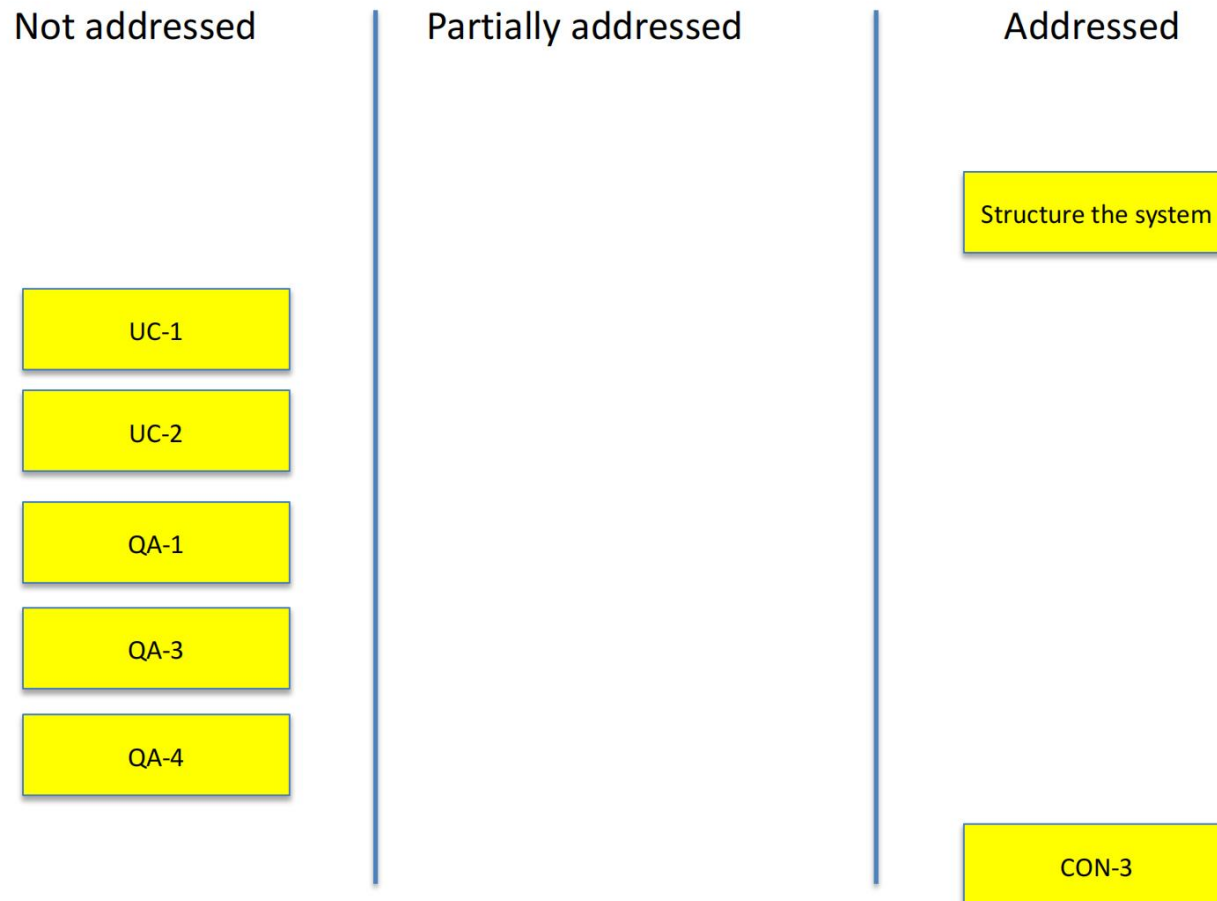
- UC-1



Attribute-Driven Design 3.0 (ADD 3.0)

Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

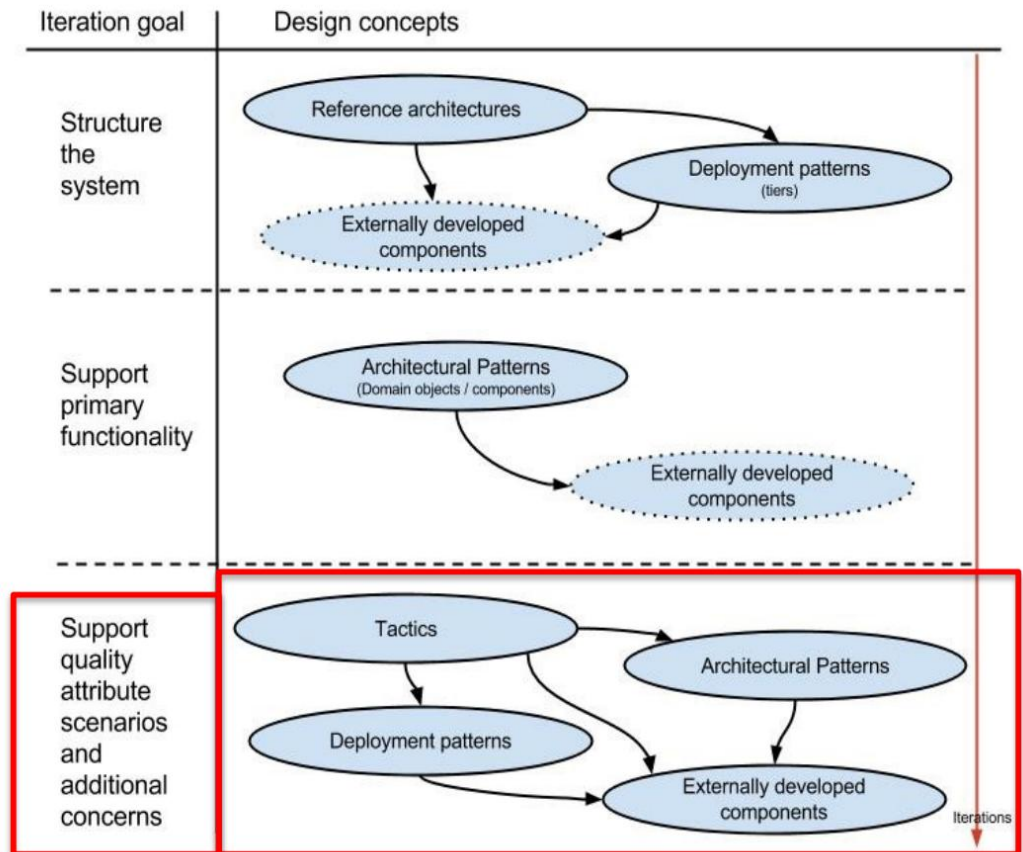
- Design Kanban board



Attribute-Driven Design 3.0 (ADD 3.0)

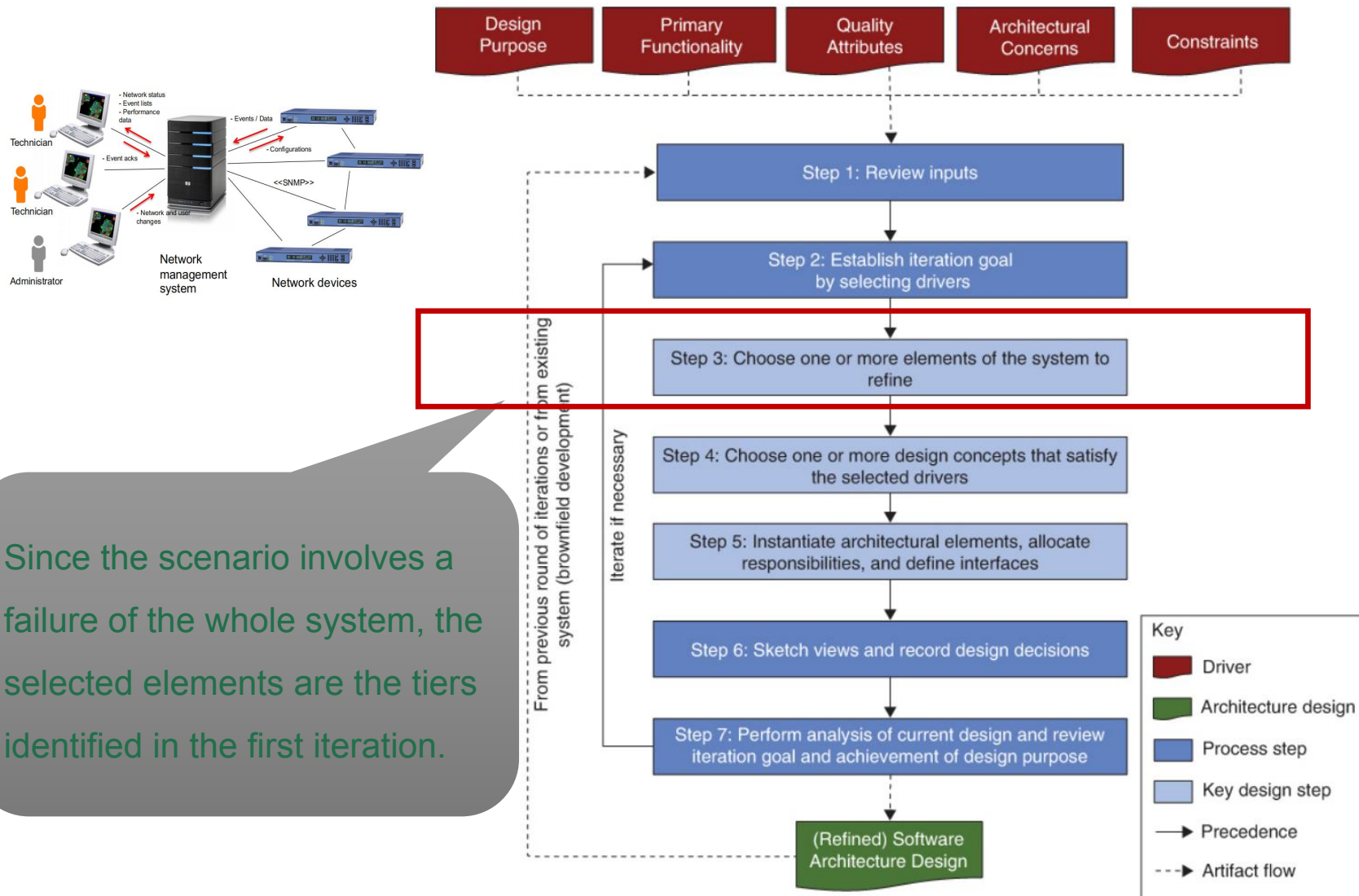
Step 2: Establish the Iteration Goal by Selecting Drivers

- **ADD : Iteration 3**
- Iteration goal:
- Address quality attribute QA-3: A failure occurs in the network management system during operation. The system resumes operation in less than 30 seconds.



Attribute-Driven Design 3.0 (ADD 3.0)

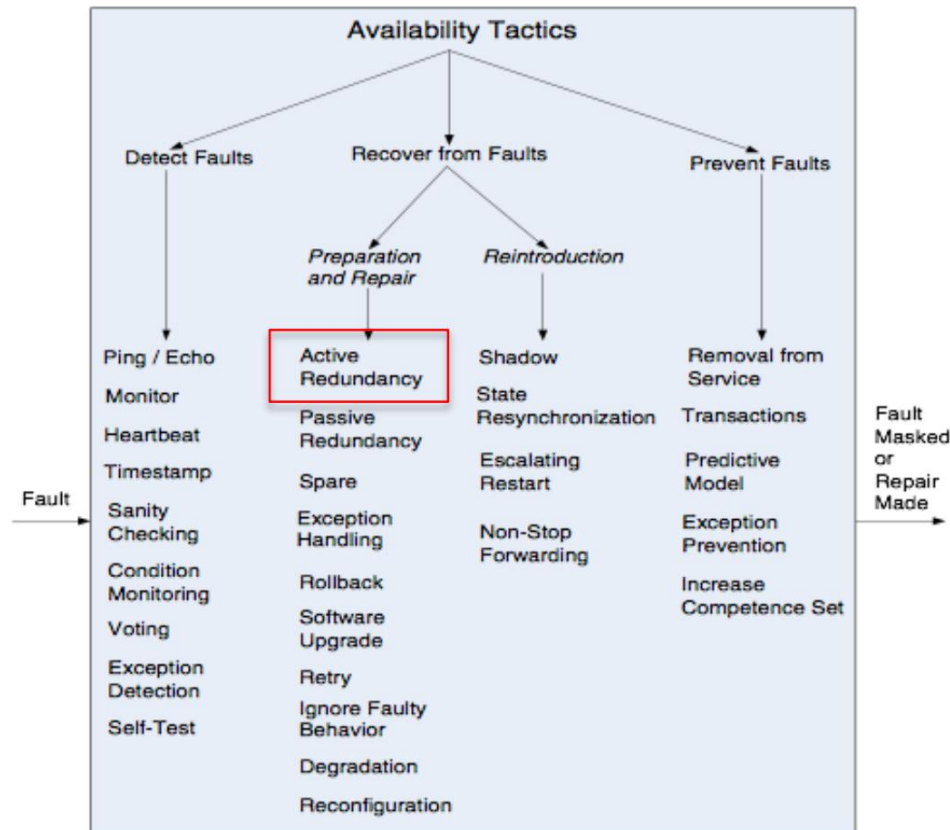
Step 3: Choose One or More Elements of the System to Refine



Attribute-Driven Design 3.0 (ADD 3.0)

Step 4: Selection of Design Concepts

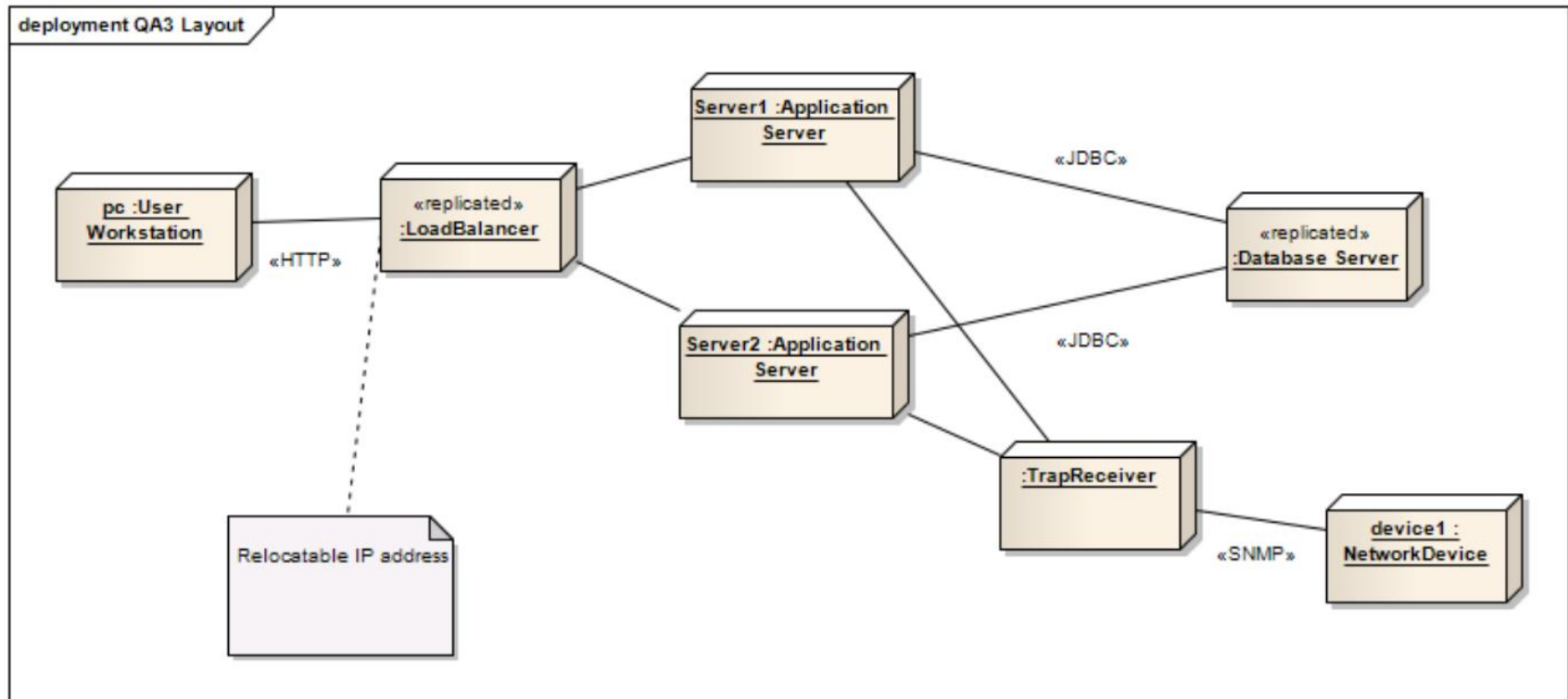
- It is recommended to start with tactics and from there go to patterns or technologies



Design decision	Rationale
Introduce active redundancy by replicating the application server and other critical components such as the database	By replicating the critical components, the system can withstand the failure of one of the replicated elements without affecting functionality.
Implement redundancy using Apache + mod_proxy	Apache serves as the point of entry, mod_proxy serves as load balancer

Attribute-Driven Design 3.0 (ADD 3.0)

Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces



Element	Responsibility	Properties
TrapReceiver	Receive traps from network devices, convert them into events and put these events into a message queue	Framework = SNMP4J
...	...	...

Attribute-Driven Design 3.0 (ADD 3.0)

Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

- Design Kanban board

Not addressed

QA-1

QA-3

QA-4

Partially addressed

Addressed

Structure the system

UC-1

UC-2

More decisions need to be made

CON-3

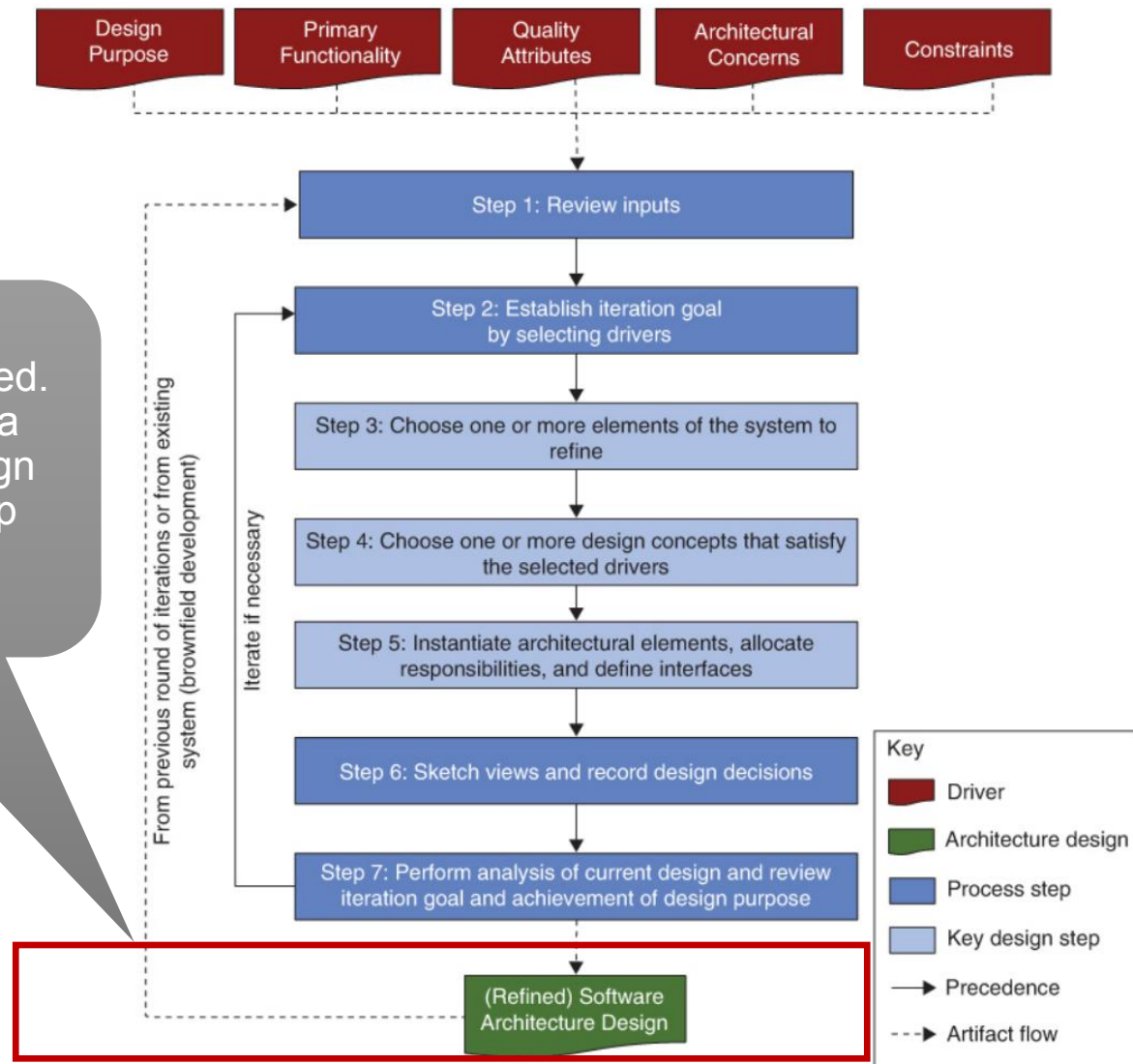
Attribute-Driven Design 3.0 (ADD 3.0)

Design Process Termination Criteria

- The design process continues across several iterations:
 - until design decisions have been made for **all of the driving architectural requirements** (design goal reached); or
 - until **the most important technical risks** have been mitigated;
 - until the **time allotted** for architecture design is consumed (not very desirable!).

Attribute-Driven Design 3.0 (ADD 3.0)

Iterate If Necessary



Applying ADD to Different System Contexts

Tailoring Design Approaches by System Type



Categories of designing software systems

- 01 Design of a greenfield system for a mature (i.e., well-known) domain.
- 02 design of a greenfield system for a domain that is novel (i.e., a domain that has a less established infrastructure and knowledge base)
- 03 design for making changes to an existing system (brownfield)

Applying ADD to Different System Contexts

Designing Greenfield Systems in Mature Domains

- **Greenfield System in Mature Domains**
 - Design for “from-scratch” systems in well-known domains like desktop, mobile, web enterprise apps and microservices.
- **Initial Iteration Goal**
 - Establish overall system structure. Select reference architectures, patterns, frameworks. Prioritize non-functional constraints and quality attributes.
- **Next Iteration Goal**
 - Identify structures for primary functionality. Allocate use cases to elements, decompose reference architecture elements, plan team assignments.
- **Subsequent Iterations**
 - Refine structures for remaining drivers. Use tactics, patterns, components, and best practices (modularity, low coupling).
- **Roadmap Benefits**
 - Guides initial design, aids early estimation. Follow a defined roadmap using known components to reduce risk and support quality goals from the start.

Applying ADD to Different System Contexts

Designing Greenfield Systems in Novel Domains

Start Without Reference Architecture



No existing models or reusable components—design begins from first principles.

Use Prototyping and General Concepts



Guide design using patterns, tactics, and quality attribute-focused prototypes.

Address Emerging Challenges



Focus on performance, scalability, security; novel domains need flexible iteration goals.

Applying ADD to Different System Contexts

Designing for an Existing System (Brownfield)

Multiple Redesign Goals

- ! Brownfield architecture design is often driven by maintenance, refactoring, or resolving quality issues.

Understand Existing Architecture First



Identifying system elements and their interactions is key before decomposition or iteration planning.

Design Iterations Post-Assessment



Once architecture is understood, ADD steps resemble greenfield design, targeting new drivers incrementally.

Applying ADD to Different System Contexts

Design to Replace a Legacy Application

Legacy Tech and Technical Debt



Old systems may rely on obsolete technology or have unmanageable technical debt.

Use the Strangler Fig Pattern



Introduce a **proxy** to **route calls** to legacy systems while gradually replacing components.

Iterative Replacement



Design goals in each ADD iteration target specific services or components to transition.



Documenting Software Architectures



Architecture Documentation

- Even the best architecture will be useless if the people who need
 - it do not know what it is;
 - cannot understand it well enough to use, build, or modify it;
 - misunderstand it and apply it incorrectly.
- All of the effort, analysis, hard work, and insightful design on the part of the architecture team will have been wasted.

Uses and Audiences

- Architecture documentation must
 - be sufficiently **transparent and accessible** to be quickly understood by new employees
 - be sufficiently **concrete** to serve as a **blueprint** for construction
 - have **enough information** to serve as a basis for **analysis**.
- Architecture documentation is both **prescriptive and descriptive**.
 - For some audiences, it prescribes what **should be true**, placing **constraints on decisions yet to be made**.
 - For other audiences, it describes what **is true**, recounting decisions **already made** about a system's design.
- Understanding **stakeholder uses** of architecture documentation is essential
 - Those uses determine the **information to capture**.

Three Uses of Architecture Documentation

- Education
 - Introducing people to the system
 - New members of the team
 - External analysts or evaluators
 - New architect
- Primary vehicle for communication among stakeholders
 - Especially architect to developers
 - Especially architect to future architect!
- Basis for system analysis and construction
 - Architecture tells implementers what to implement.
 - Each module has interfaces that must be provided and uses interfaces from other modules.
 - Documentation can serve as a receptacle for registering and communicating unresolved issues.
 - Architecture documentation serves as the basis for architecture evaluation.

Notations

- **Informal notations**
 - Views are depicted (often graphically) using **general-purpose** diagramming and editing tools
 - The semantics of the description are characterized in **natural language**
 - They cannot be formally analyzed
- **Semiformal notations**
 - Standardized **notation** that prescribes graphical elements and rules of construction
 - Lacks a complete **semantic** treatment of the meaning of those elements
 - **Rudimentary analysis** can be applied
 - **UML** is a semiformal notation in this sense.
- **Formal notations**
 - Views are described in a notation that has a **precise (usually mathematically based) semantics**.
 - Formal analysis of both **syntax and semantics** is possible.
 - **Architecture description languages (ADLs)**
 - Support **automation** through associated tools.

Choosing a Notation

- **Tradeoffs**
 - Typically, more formal notations take **more time and effort** to create and understand, but offer **reduced ambiguity** and **more opportunities for analysis**.
 - Conversely, more informal notations are **easier to create**, but they provide **fewer guarantees**.
- Different **notations** are **better (or worse)** for expressing different kinds of **information**.
 - UML class diagram will not help you reason about **schedulability**, nor will a sequence chart tell you very much about the system's **likelihood of being delivered on time**.
 - Choose your notations and representation languages **knowing the important issues** you need to capture and reason about.

Views

- Views let us **divide** a software architecture into a number of (we hope) **interesting and manageable representations** of the system.
- Principle of architecture documentation:
 - *Documenting an architecture is a matter of **documenting the relevant views** and then **adding documentation that applies to more than one view**.*

Which Views? The Ones You Need!

- Different views support different goals and uses.
- We do not advocate a particular view or collection of views.
- The views you should document depend on the uses you expect to make of the documentation.
- Each view has a cost and a benefit; you should ensure that the benefits of maintaining a view outweigh its costs.

Overview of Module Views

- **Elements**

- Modules, which are implementation units of software that provide a coherent set of responsibilities.

- **Relations**

- *Is part of*, which defines a part/whole relationship between the submodule—the part—and the aggregate module—the whole.
- *Depends on*, which defines a dependency relationship between two modules. Specific module views elaborate what dependency is meant.
- *Is a*, which defines a generalization/specialization relationship between a more specific module—the child—and a more general module—the parent.

Overview of Module Views

- **Constraints**

- Different module views may impose specific topological constraints, such as limitations on the visibility between modules.

- **Usage**

- Blueprint for construction of the code
- Change-impact analysis
- Planning incremental development
- Requirements traceability analysis
- Communicating the functionality of a system and the structure of its code base
- Supporting the definition of work assignments, implementation schedules, and budget information
- Showing the structure of information that the system needs to manage

Module Views

It is unlikely that the documentation of any software architecture can be complete without at least one module view.

Overview of C&C Views

- **Elements**

- *Components*. Principal processing units and data stores. A component has a set of *ports* through which it interacts with other components (via connectors).
- *Connectors*. Pathways of interaction between components. Connectors have a set of roles (interfaces) that indicate how components may use a connector in interactions.

- **Relations**

- *Attachments*. Component ports are associated with connector roles to yield a graph of components and connectors.
- *Interface delegation*. In some situations component ports are associated with one or more ports in an “internal” subarchitecture. The case is similar for the roles of a connector.

Overview of C&C Views

- **Constraints**

- Components can only be attached to connectors, not directly to other components.
- Connectors can only be attached to components, not directly to other connectors.
- Attachments can only be made between compatible ports and roles.
- Interface delegation can only be defined between two compatible ports (or two compatible roles).
- Connectors cannot appear in isolation; a connector must be attached to a component.

- **Usage**

- Show how the system works.
- Guide development by specifying structure and behavior of runtime elements.
- Help reason about runtime system quality attributes, such as performance and availability.

Notations for C&C Views

- UML components are good match for C&C components.



Notations for C&C Views

- UML connectors are **not rich enough** to represent many C&C connectors.
 - UML connectors **cannot have substructure, attributes, or behavioral descriptions.**
- Represent a “simple” C&C connector using a UML connector—a line.
 - Many commonly used C&C connectors have **well-known, application-independent semantics and implementations**, such as function calls or data read operations.
 - You can use a **stereotype** to denote **the type of connector.**
- **Connector roles** cannot be explicitly represented with a **UML connector.**
 - The UML connector element does not allow the inclusion of interfaces.
 - Label the connector ends and use these labels to identify role descriptions that must be documented elsewhere.
- Represent a “rich” C&C connector using a UML component, or by annotating a line UML connector with a tag that explains the meaning of the complex connector.

Overview of Allocation Views

- **Elements**

- *Software element and environmental element.*
- A software element has properties that are *required* of the environment.
- An environmental element has properties that are *provided* to the software.

- **Relations**

- *Allocated to.* A software element is mapped (allocated to) an environmental element. Properties are dependent on the particular view.

Overview of Allocation Views

- **Constraints**
 - Varies by view
- **Usage**
 - Reasoning about performance, availability, security, and safety.
 - Reasoning about distributed development and allocation of work to teams.
 - Reasoning about concurrent access to software versions.
 - Reasoning about the form and mechanisms of system installation.

Quality Views

- A *quality view* can be tailored for specific stakeholders or to address specific concerns.
- A quality views is formed by extracting the relevant pieces of structural views and packaging them together.

Quality Views: Examples

- **Security view**

- Show the components that have some security role or responsibility, how those components communicate, any data repositories for security information, and repositories that are of security interest.
- The view's context information would show other security measures (such as physical security) in the system's environment.
- The behavior part of a security view
 - Show how the operation of security protocols and where and how humans interact with the security elements.
 - Capture how the system would respond to specific threats and vulnerabilities.

- **Communications view**

- Especially helpful for systems that are globally dispersed and heterogeneous.
- Show all of the component-to-component channels, the various network channels, quality-of-service parameter values, and areas of concurrency.
- Used to analyze certain kinds of performance and reliability (such as deadlock or race condition detection).
- The behavior part of this view could show (for example) how network bandwidth is dynamically allocated.

Quality Views: Examples

- ***Exception or error-handling view***
 - Could help illuminate and draw attention to error reporting and resolution mechanisms.
 - Show how components detect, report, and resolve faults or errors.
 - It would help identify the sources of errors and appropriate corrective actions for each.
- ***Reliability view***
 - Models mechanisms such as replication and switchover.
 - Depicts timing issues and transaction integrity.
- ***Performance view***
 - Shows those aspects of the architecture useful for inferring the system's performance.
 - Show network traffic models, maximum latencies for operations, and so forth.

Choosing the Views

- You can determine **which** views are required, **when** to create them, and **how much detail** to include if you know the following:
 - What people, and with what skills, are available
 - Which standards you have to comply with
 - What budget is on hand
 - What the schedule is
 - What the information needs of the important stakeholders are
 - What the driving quality attribute requirements are
 - What the approximate size of the system is

*At a minimum, expect to have at least **one module view**, at least **one C&C view**, and for larger systems, at least one **allocation view** in your architecture document.*

Method for Choosing the Views

- **Step 1. Build a stakeholder/view table.**
 - **Rows:** List the **stakeholders** for your project's software architecture documentation
 - **Columns:** Enumerate the **views** that apply to your system.
 - Use the structures and views, and **the views that your design work in ADD** has suggested as a starting list of candidates.
 - Include the **views or view sketches** you have as a result of your design work so far.
 - Some views (such as **decomposition, uses, and work assignment**) apply to every system, while others (**various C&C views, the layered view**) only apply to some systems.
 - Fill in each cell to describe **how much information** the stakeholder requires from the view: **none, overview only, moderate detail, or high detail.**

Method for Choosing the Views

- **Step 2. Combine views to reduce their number**
 - Look for *marginal views* in the table; those that require only an overview, or that serve very few stakeholders.
 - *Combine each marginal view with another view* that has a stronger constituency.
 - These views often *combine naturally*:
 - *Various C&C views*. Because C&C views all show runtime relations among components and connectors of various types, they tend to combine well.
 - *Deployment view with either SOA or communicating-processes views*. An SOA view shows services, and a communicating-processes view shows processes. In both cases, these are components that are deployed onto processors.
 - *Decomposition view and any of work assignment, implementation, uses, or layered views*. The decomposed modules form the units of work, development, and uses. In addition, these modules populate layers.

Method for Choosing the Views

- **Step 3. Prioritize and stage.**
 - The **decomposition view (one of the module views)** is a particularly helpful view to release early.
 - **High-level decompositions** are often easy to design
 - The project manager can start to staff development teams, put training in place, determine which parts to outsource, and start producing budgets and schedules.
 - You **don't have to satisfy all the information needs** of all the stakeholders to the fullest extent.
 - Providing **80 percent** of the information goes a long way, and this might be “good enough” so that the stakeholders can do their job.
 - Check with the stakeholder if a subset of information would be sufficient.
 - You **don't have to complete one view before starting another.**
 - People can make progress with overview-level information
 - A **breadth-first** approach is often the best.

Building the Documentation Package

- Documentation package consists of
 - Views
 - Documentation beyond views

Documenting a View

- **Section 1: The Primary Presentation.**
 - The *primary presentation* shows the elements and relations of the view.
 - The primary presentation should contain the information you wish to convey about the system—in the vocabulary of that view.
 - The primary presentation is most often graphical.
 - It might be a diagram you've drawn in an informal notation using a simple drawing tool, or it might be a diagram in a semiformal or formal notation imported from a design or modeling tool that you're using.
 - If your primary presentation is graphical, make sure to include a key that explains the notation.
 - Lack of a key is the most common mistake that we see in documentation in practice.
 - Occasionally the primary presentation will be textual, such as a table or a list.
 - If that text is presented according to certain stylistic rules, these rules should be stated or incorporated by reference, as the analog to the graphical notation key.

Documenting a View

- **Section 2: The Element Catalog.**
 - The *element catalog* details at least those elements depicted in the primary presentation.
 - For instance, if a diagram shows elements A, B, and C, then the element catalog needs to explain what A, B, and C are.
 - If elements or relations relevant to this view were omitted from the primary presentation, they should be introduced and explained in the catalog.
 - Parts of the catalog:
 - *Elements and their properties*. This section names each element in the view and lists the properties of that element.
 - *Relations and their properties*. Each view has specific relation types that it depicts among the elements in that view.
 - *Element interfaces*. This section documents element interfaces.
 - *Element behavior*. This section documents element behavior that is not obvious from the primary presentation.

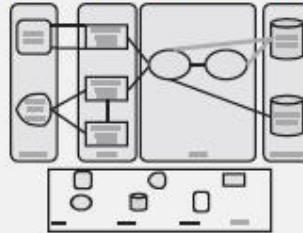
Documenting a View

- **Section 3: Context Diagram.**
 - A *context diagram* shows how the system or portion of the system depicted in this view relates to its environment.
 - The purpose of a context diagram is to depict the scope of a view.
 - Entities in the environment may be humans, other computer systems, or physical objects, such as sensors or controlled devices.
- **Section 4: Variability Guide.**
 - A *variability guide* shows how to exercise any variation points that are a part of the architecture shown in this view.
- **Section 5: Rationale.**
 - *Rationale* explains why the design reflected in the view came to be.
 - The goal of this section is to explain why the design is as it is and to provide a convincing argument that it is sound.
 - The choice of a pattern in this view should be justified here by describing the architectural problem that the chosen pattern solves and the rationale for choosing it over another.

View Template

Template for a View

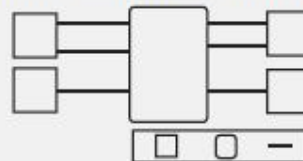
Section 1. Primary Presentation



Section 2. Element Catalog

- Section 2.A. Elements and Their Properties
- Section 2.B. Relations and Their Properties
- Section 2.C. Element Interfaces
- Section 2.D. Element Behavior

Section 3. Context Diagram



Section 4. Variability Guide

Section 5. Rationale

Documenting Information Beyond Views

- **Document control information.**
 - List the issuing organization, the current version number, date of issue and status, a change history, and the procedure for submitting change requests to the document.
 - Usually captured in the front matter

Documenting Information Beyond Views

- **Section 1: Documentation Roadmap.**
 - The documentation map tells the reader **what information** is in the documentation and **where to find it**.
 - *Scope and summary*. Explain the **purpose** of the document and briefly summarize **what is covered**.
 - *How the documentation is organized*. For each section in the documentation, give a short synopsis of the information that can be found there.
 - *View overview*. Describes the views that the architect has included in the package. For each view:
 - The **name of the view** and **what pattern it instantiates**, if any.
 - A description of the **view's element types, relation types, and property types**.
 - A description of **language, modeling techniques, or analytical methods** used in constructing the view.
 - *How stakeholders can use the documentation*.
 - This section shows how **various stakeholders might use** the documentation to help address their concerns.
 - Include **short scenarios**, such as “A maintainer wishes to know the units of software that are likely to be changed by a proposed modification.”
 - To be compliant with ISO/IEC 42010-2007, you must consider the **concerns** of at least users, acquirers, developers, and maintainers.

Documenting Information Beyond Views

- **Section 2: How a View Is Documented.**
 - Explain the **standard organization** you're using to document views—either the one described in this part or one of your own.
- **Section 3: System Overview.**
 - Short prose description of the **system's function, its users, and any important background or constraints.**
 - Provides your readers with a **consistent mental model of the system and its purpose.**
 - This might be a pointer to your project's **concept-of-operations document** for the system.

Documenting Information Beyond Views

- **Section 4: Mapping Between Views.**
 - Helping a reader understand the **associations between views** will help that reader gain a powerful insight into how the architecture works as a unified conceptual whole.
 - The associations between elements across views in an architecture are, in general, **many-to-many**.
 - **View-to-view associations can be captured as tables.**
 - The table should name the correspondence between the elements across the two views.
 - Examples
 - **“is implemented by”** for mapping from a component-and-connector view to a module view
 - **“implements”** for mapping from a module view to a component-and-connector view
 - **“included in”** for mapping from a decomposition view to a layered view

Documenting Information Beyond Views

- **Section 5: Rationale.**
 - Documents the **architectural decisions** that apply to more than one view.
 - Documentation of **background or organizational constraints or major requirements** that led to decisions of system-wide import.
 - Decisions about which **fundamental architecture patterns** are used.
- **Section 6: Directory.**
 - Set of **reference material** that helps readers **find more information quickly**.
 - Index of terms
 - Glossary
 - Acronym list.

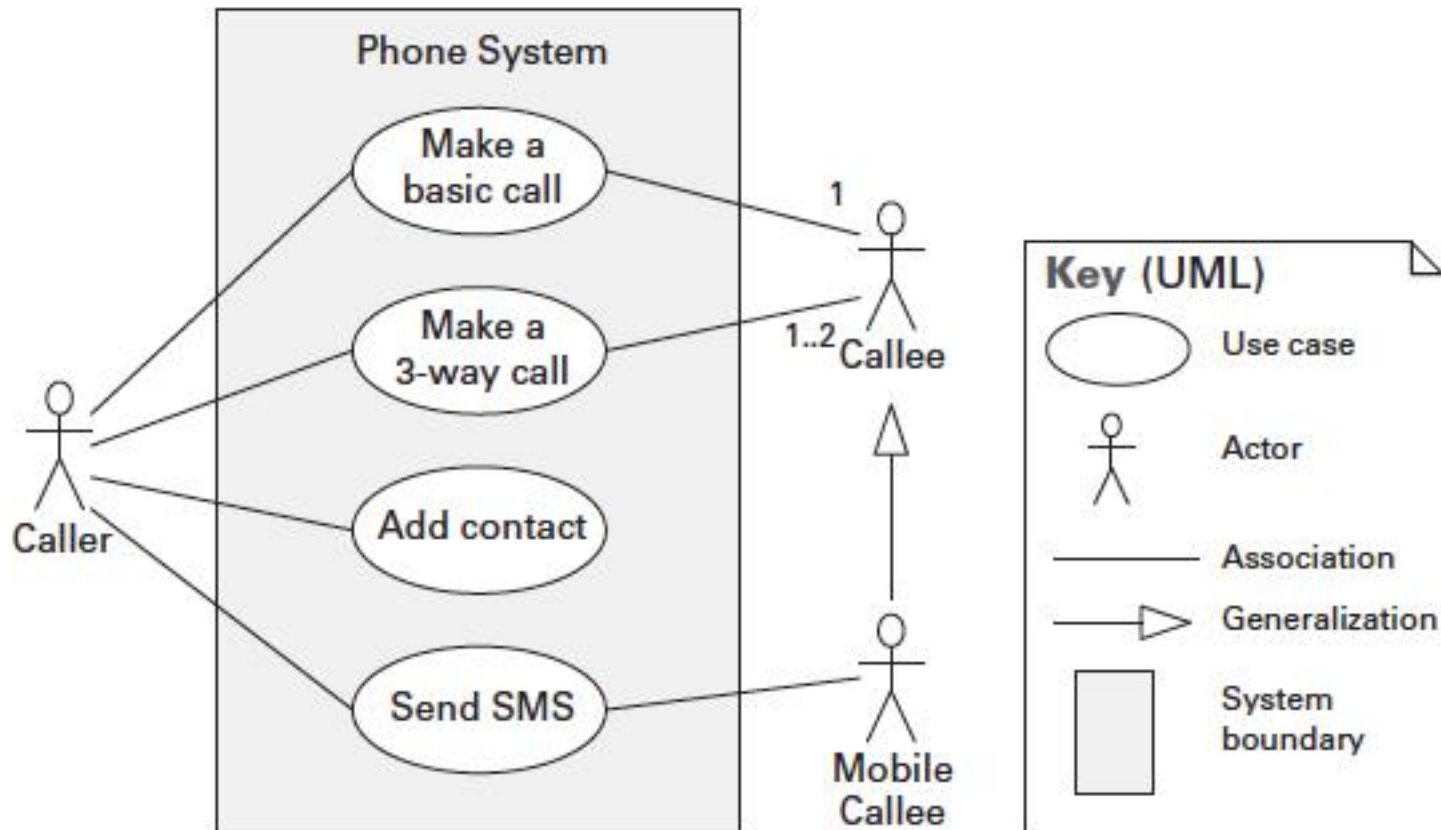
Documenting Behavior

- Behavior documentation complements each views by describing how architecture **elements** in that view **interact with each other**.
- Behavior documentation enables reasoning about
 - a system's **potential to deadlock**
 - a system's ability to **complete a task in the desired amount of time**
 - **maximum memory consumption**
 - and more
- Behavior has **its own section** in our view template's element catalog.

Notations for Documenting Behavior

- Trace-oriented languages
 - *Traces* are sequences of activities or interactions that describe the system's response to a specific stimulus when the system is in a specific state.
 - A trace describes a particular sequence of activities or interactions between structural elements of the system.
 - Examples
 - use cases
 - sequence diagrams
 - communication diagrams
 - activity diagrams
 - message sequence charts
 - timing diagrams
 - Business Process Execution Language

Use Case Diagram



Use Case Description

Name: Make a basic call

Description: Making a point-to-point connection between two phones.

Primary actors: Caller

Secondary actors: Callee

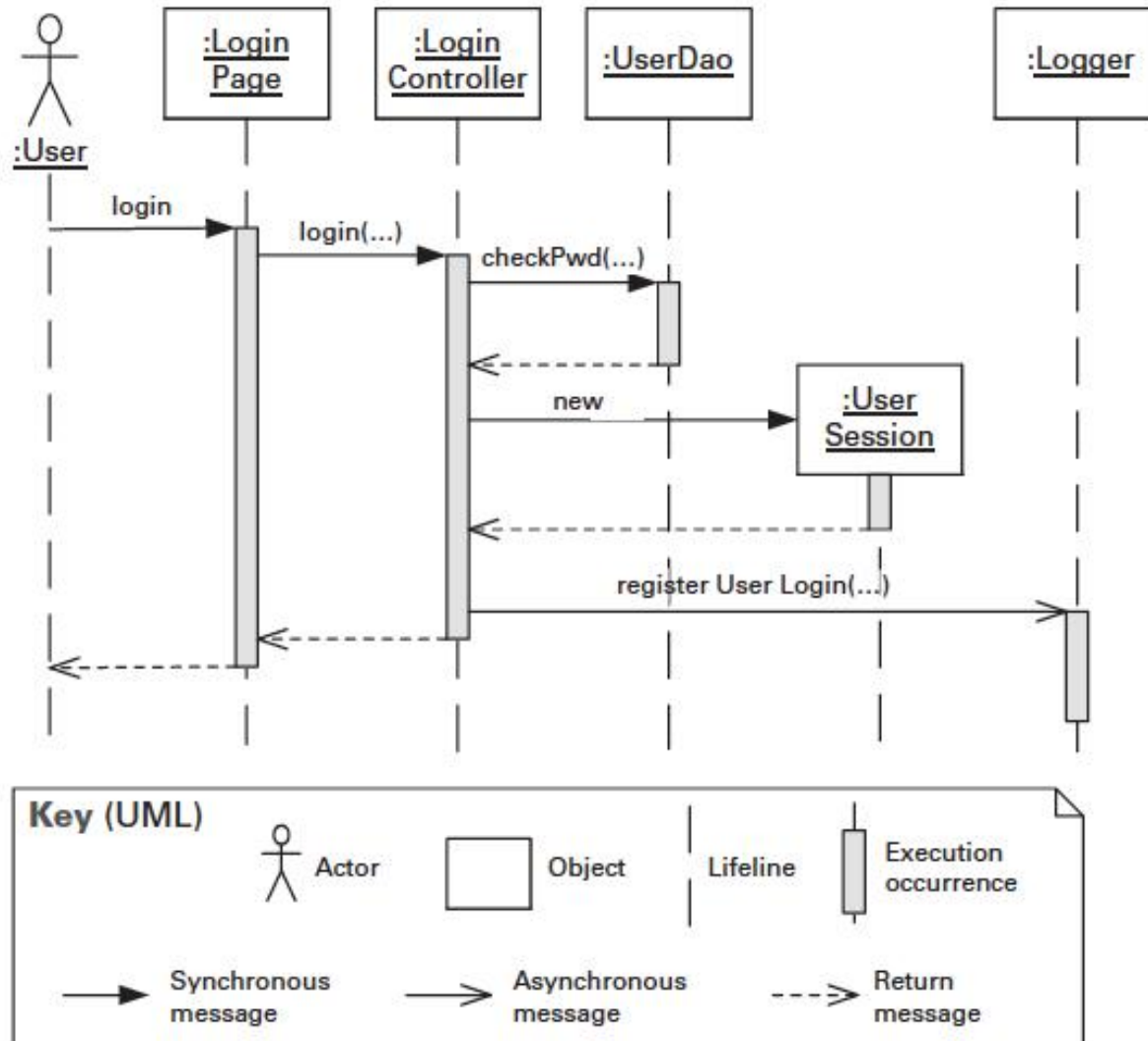
Flow of events:

The use case starts when a caller places a call via a terminal, such as a cell phone. All terminals to which the call should be routed then begin ringing. When one of the terminals is answered, all others stop ringing and a connection is made between the caller's terminal and the terminal that was answered. When either terminal is disconnected—someone hangs up—the other terminal is also disconnected. The call is now terminated, and the use case is ended.

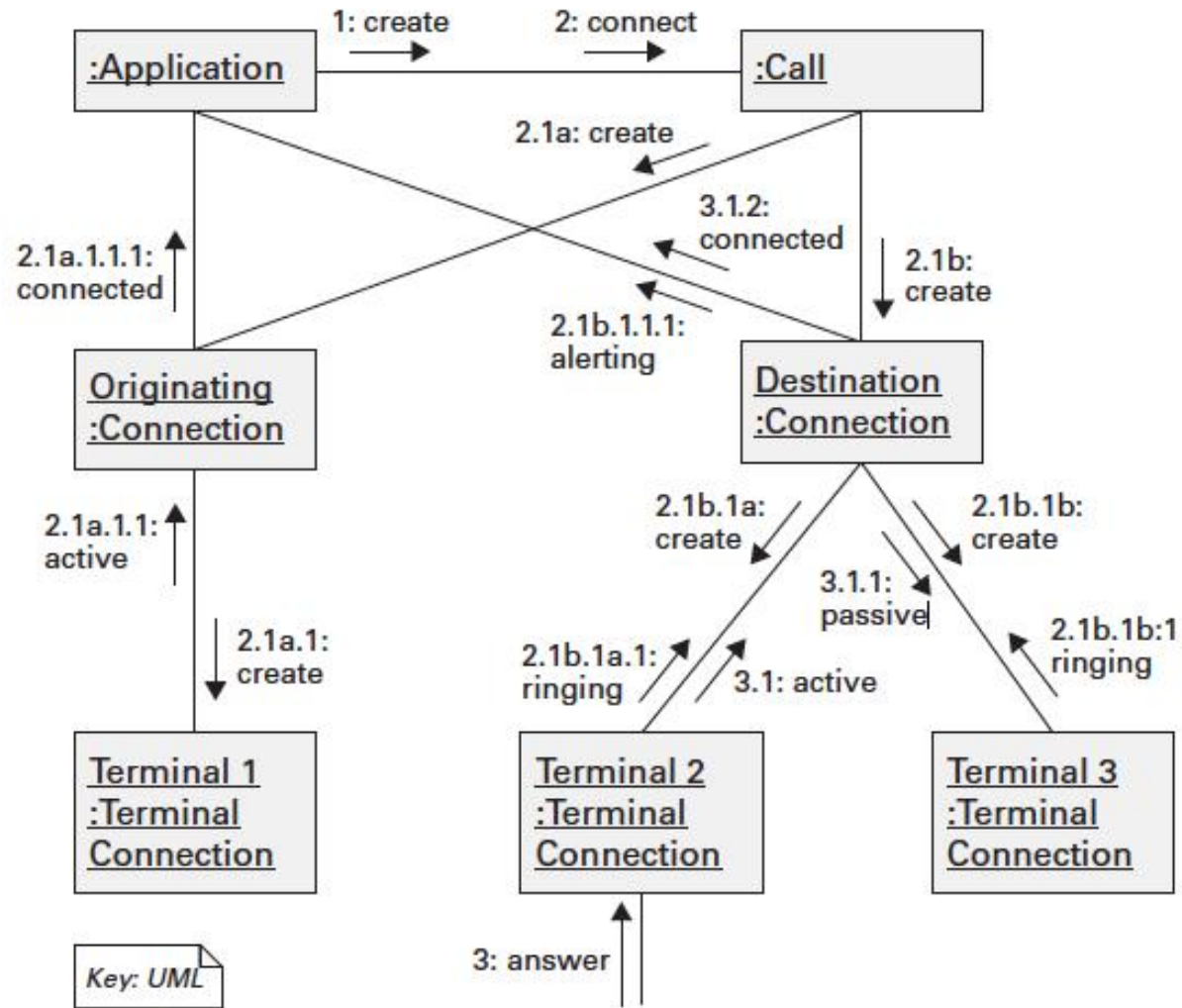
Exceptional flow of events:

The caller can disconnect, or hang up, before any of the ringing terminals has been answered. If this happens, all ringing terminals stop ringing and are disconnected, ending the use case.

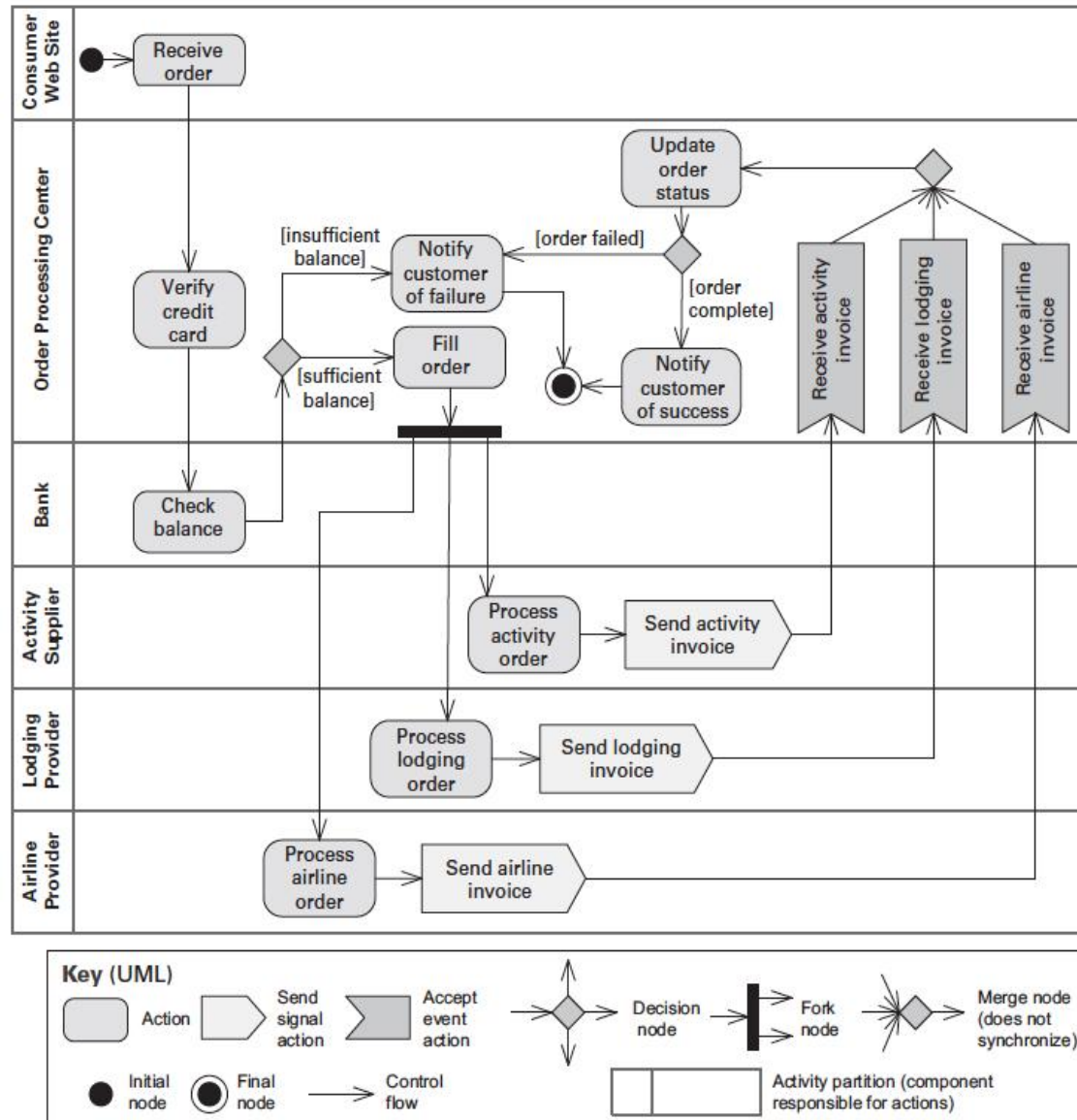
Sequence Diagram



Communication Diagram



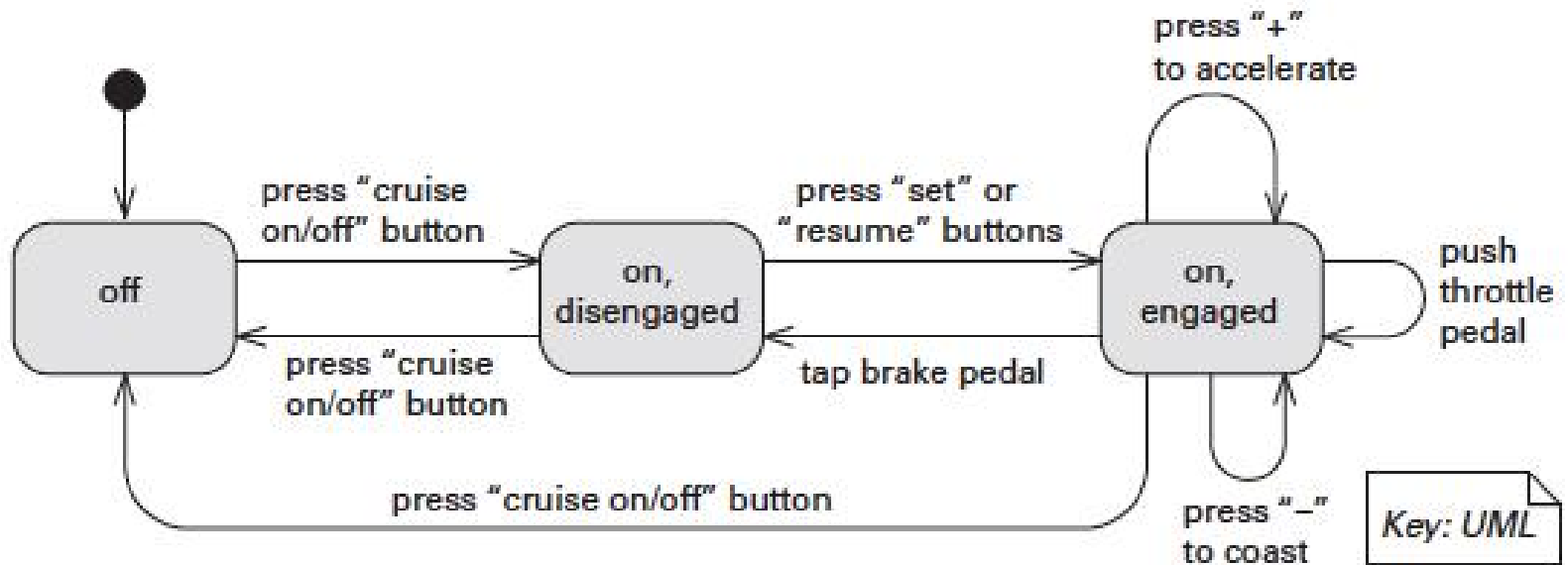
Activity Diagram



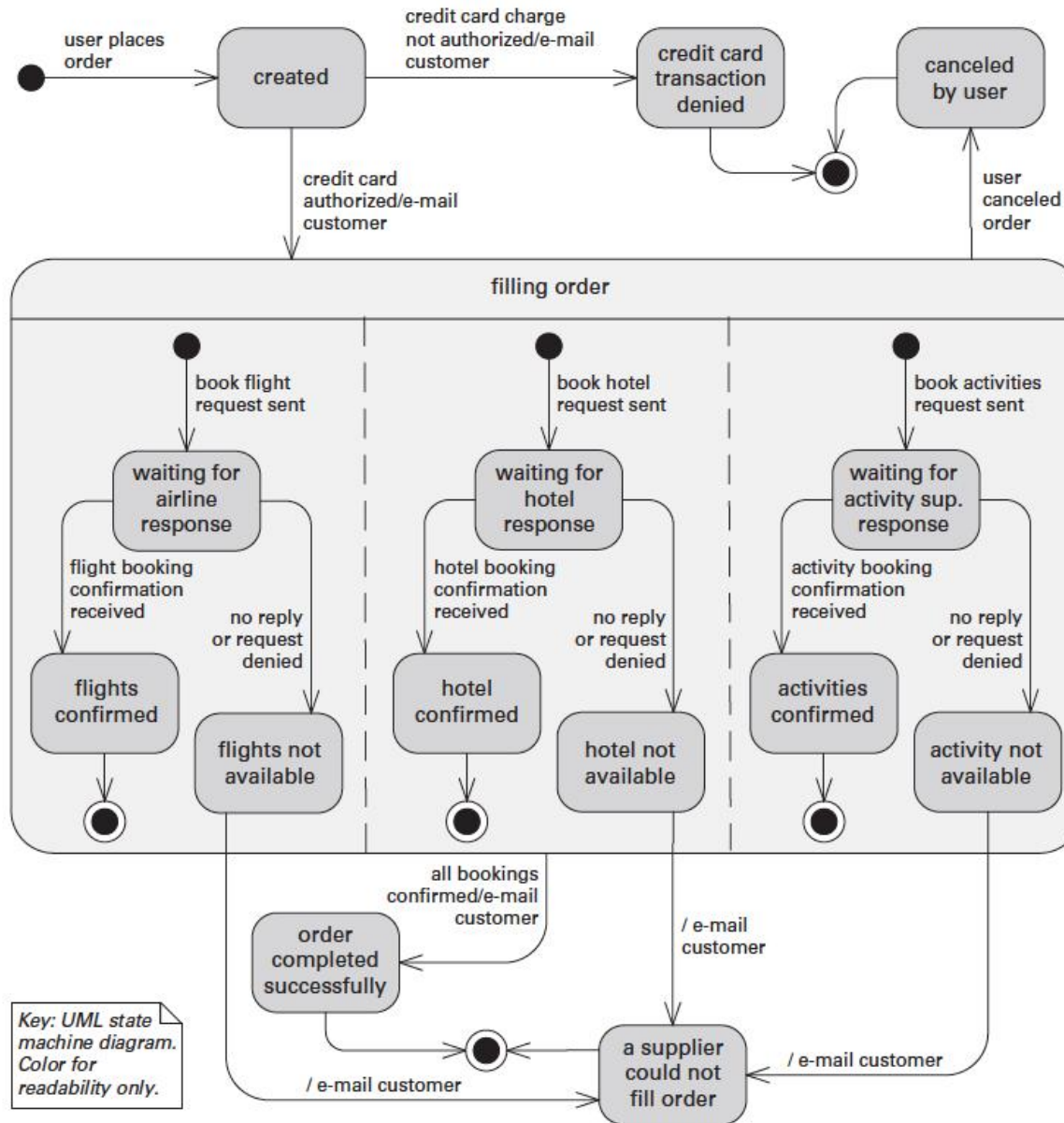
Notations for Documenting Behavior

- Comprehensive languages
 - *Comprehensive models* show the complete behavior of structural elements.
 - Given this type of documentation, it is possible to infer all possible paths from initial state to final state.
 - The state machine formalism represents the behavior of architecture elements because each state is an abstraction of all possible histories that could lead to that state.
 - State machine languages allow you to complement a structural description of the elements of the system with constraints on interactions and timed reactions to both internal and environmental stimuli.

State Machine



State Machine



Documenting Architectures That Change Faster Than You Can Document Them

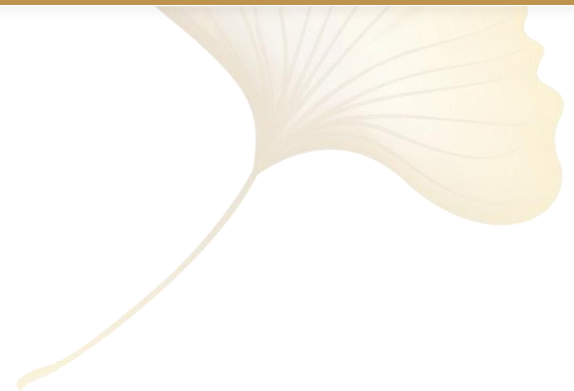
- An architecture that **changes at runtime**, or as a result of a **high-frequency release-and-deploy cycle**, change much **faster** than the documentation cycle.
- **Nobody will wait** until a new architecture document is produced, reviewed, and released.
- In this case:
 - Document **what is true about all versions** of your system. Record those **invariants** as you would for any architecture. This may make your documented architecture **more a description of constraints or guidelines** that any compliant version of the system must follow.
 - Document **the ways the architecture is allowed to change**. This will usually mean **adding new components and replacing components with new implementations**. The place to do this is called the **variability guide**.

Documenting Architecture in an Agile Development Project

- Adopt a **template or standard organization** to capture your **design decisions**.
- Plan to document a **view** if (but only if) it has a **strongly identified stakeholder constituency**.
- Fill in the sections of the **template for a view**, and for **information beyond views**, when (and in whatever order) the information becomes available. But **only do this if writing down this information will make it easier (or cheaper or make success more likely)** for someone downstream doing their job.
- Don't worry about creating an architectural design document and then a finer-grained design document. **Produce just enough design information to allow you to move on to code**.
- Don't feel obliged to fill up all sections of the template, and certainly **not all at once**. Write **"N/A"** for the **sections for which you don't need to record** the information (perhaps because you will convey it orally).
- Agile teams sometimes **make models in brief discussions** by the **whiteboard**. **Take a picture** and use it as the primary presentation.



Summary



Summary

- Architecture design transforms drivers into structures.
- Architectural drivers include functional requirements, quality attributes and constraints but also objectives, concerns and the type of system.
- ADD is a method that structures architecture design so it may be performed systematically.
- Design concepts are building blocks from which the design is created. There are several important types: Reference Architectures, Deployment patterns, Architectural Patterns, Tactics, and Externally developed components such as frameworks.
- ADD can be performed in an agile way by using initial documentation (sketches) and a design kanban board to track design advancement

Summary

- You should understand the uses to which the writing is to be put and the audience for the writing.
- Architectural documentation serves as a means for communication among various stakeholders, not only up the management chain and down to the developers but also across to peers.
- An architecture is a complicated artifact, best expressed by focusing on views.
- You must choose the views to document, must choose the notation to document these views, and must choose a set of views that is both minimal and adequate.
- You must document not only the structure of the architecture but also the behavior.



谢 谢

诚耀百廿 雄创一流

